The Imitation Game: When LLMs Learn to Reason Like Programs via Code-Centric Reasoning Data Synthesis

Jinyang Zhang[1], Weibin Liao[1], Keqin Bao[2], Sihang Li[2], Shaobo Wang[3], Muyang Ye[4], Hongxin Ding[1], Yue Fang[1], Tianyi Tang[2], Fei Huang[2], Kexin Yang[2*], Xingzhang Ren[2], Dayiheng Liu[2]

[1]School of Computer Science, Peking University

[2]Alibaba Group [3]EPIC Lab, SJTU

[4]College of Computer Science and Technology, Zhejiang University.

✉ jinyangzhang25@stu.pku.edu.cn, yangkexin.ykx@alibaba-inc.com

Abstract

Large Language Models (LLMs) excel at programming tasks but frequently fail at deterministic, fine-grained reasoning in natural language, relying heavily on semantic approximations rather than robust symbolic execution. To bridge this gap, we propose MIMIC, a framework that leverages executable code as a rigorous medium for reasoning data synthesis. MIMIC fundamentally transforms algorithms into verifiable reasoning trajectories through narrative fusion, code-guided test synthesis, and dynamic code instrumentation. Crucially, these explicit intermediate execution states naturally form a Code-Instrumented Reward (CIR), providing dense, high-fidelity process supervision for reinforcement learning without external reward models. Extensive evaluations reveal that models trained via SFT and GRPO on our synthesized dataset achieve substantial, consistent gains. Our method significantly elevates accuracy across general reasoning, complex mathematical benchmarks, and fine-grained deterministic tasks, demonstrating that the procedural rigor of executable code can effectively unlock and enhance the generalized reasoning capabilities of LLMs. Our code and data are available at https://github.com/zjy1298/MIMIC.

1 Introduction

> *"A person who is not good at calculating can still become a first-rate mathematician, whereas someone who has no sense for mathematics at all will at best become a great calculator."* — Novalis

Large Language Models (LLMs) (Xu et al., 2025; Liao et al., 2025; Zhang et al., 2026b; Ding et al., 2025) have demonstrated remarkable proficiency in formal environments, frequently surpassing human baselines on competitive programming benchmarks and code synthesis tasks (Liu et al., 2024; Hurst et al., 2024; Zhang et al., 2025a). Yet, a stark paradox persists: the very same models that can author complex dynamic programming solutions routinely fail on natural language tasks requiring rudimentary, fine-grained symbolic fidelity (Zhou et al., 2024; Yu et al., 2024b). They struggle with deterministic operations that any trivial program executes flawlessly, such as comparing the numerical values 9.9 and 9.11 (Huang et al., 2026; Zhang et al., 2026a), tracking sequential state transitions, or counting the exact occurrences of a character in a word (as illustrated in Figure 1).

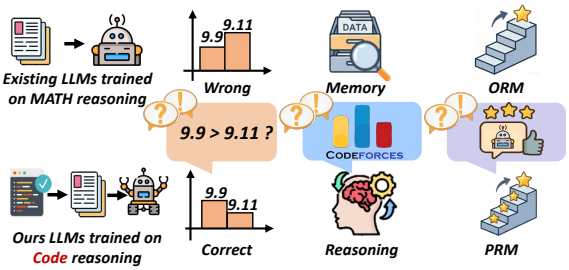


Figure 1: The paradox of LLMs: while they excel at complex programming logic, they often fail at basic deterministic natural language reasoning. MIMIC bridges this gap by leveraging executable code to generate hallucination-mitigated reasoning trajectories.

This asymmetry suggests that current "reasoning" capabilities in LLMs are largely driven by approximate semantic pattern-matching rather than internalized symbolic execution (Dziri et al., 2023; Mirzadeh et al., 2025). While a dominant line of research attempts to enhance reasoning purely within the natural language domain (e.g., via chain-of-thought prompting or verifiers) to improve coding capabilities (Chen et al., 2022a; Wei et al., 2022; Chen et al., 2024; Liao et al., 2026b), the reverse direction—leveraging a model's robust *coding* capabilities to bootstrap its *general reasoning*—remains underexplored (Yang et al., 2025b). The underlying premise is that accepted, executable code inherently encapsulates the rigorous, step-by-

*Corresponding author. Work done when Jinyang Zhang is intern at Qwen Team, Alibaba Group.

Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing, pages 35579–35622
October 24-29, 2026 ©2026 Association for Computational Linguistics

step logical deduction that LLMs lack in prose.

Unlike prior work that mainly studies how to transfer code or execution signals into reasoning, we argue that the surface form of competitive programming itself induces shortcut memorization, causing models to recall programming templates rather than perform transferable natural-language reasoning. This motivates MIMIC as a de-biasing framework: it removes programming-specific cues while preserving the underlying executable reasoning process.

However, translating competitive programming knowledge into generalized reasoning training data introduces significant challenges. Our preliminary investigations reveal that simply providing models with raw programming problems triggers a superficial "recall illusion." (Janiak et al., 2025; Sun et al., 2026) Instead of reasoning through the logic, models recognize the structural formatting of competitive programming tasks and attempt to reconstruct memorized code templates from their pre-training corpora, short-circuiting the actual reasoning process (Guo et al., 2024; Wang et al., 2026). Furthermore, standard data synthesis pipelines often rely on stronger LLMs to fabricate reasoning trajectories, which inevitably injects silent factual errors and computational hallucinations into the intermediate steps (Gomez-Villa et al., 2022).

This leads to our primary **research question**: *How can we systematically distill the deterministic, procedural rigor of executable code into natural language reasoning trajectories, while simultaneously preventing models from bypassing reasoning via memorized coding patterns?*

To address this question, we propose MIMIC, **M**ining **I**ntelligence from **M**achine-executed **I**nstrumented **C**ode, an end-to-end synthesis framework designed to bridge the "Coding → Reasoning" gap. MIMIC guarantees computational correctness by strictly dividing labor: *logic is articulated by the language model, but computation is guaranteed by deterministic code execution*. Specifically, our framework solves the aforementioned challenges through three core methodological innovations:

1. **Preventing Shortcut Memorization via Narrative Fusion:** To break the coding-problem surface patterns that trigger recall, MIMIC removes all programmatic artifacts from source problems and fuses their core mathematical logic with diverse, real-world corpora. This transforms algorithmic challenges into self-contained, realistic natural-language exam questions.
2. **Ensuring Generalizable Coverage via Code-Guided Synthesis:** Rather than relying on blind random sampling, MIMIC directly analyzes the control-flow boundaries of the accepted code to synthesize targeted adversarial and guided-random test cases. This ensures that the generated reasoning scenarios probe deep logical branches rather than superficial scale.
3. **Eliminating Hallucinations via Trace-Grounded CoT:** To generate verifiable reasoning trajectories, MIMIC dynamically instruments the accepted code with semantic probes. Executing this instrumented code yields a deterministic sequence of intermediate states. The natural language chain-of-thought is then strictly anchored to these explicit execution traces, ensuring every quantitative claim is grounded in computational reality. These intermediate states further serve as a natural source of *process-level reward signals* during reinforcement learning, providing denser supervision than binary outcome rewards without requiring a separate reward model.

By solving these structural issues, MIMIC produces high-quality, hallucination-mitigated reasoning data that systematically improves LLM performance across both general reasoning and mathematical benchmarks, demonstrating a profound bidirectional alignment between code understanding and natural-language reasoning.

In summary, our main contributions are threefold:

- **Conceptual Insight:** We identify the critical gap between LLMs' strong formal coding capabilities and their failures in natural language reasoning. We expose the "recall illusion" and demonstrate how deterministic code execution can serve as a reliable anchor for true reasoning.
- **Methodology:** We propose MIMIC that translates code into hallucination-mitigated reasoning trajectories. Furthermore, we introduce a *Code-Instrumented Reward* that leverages execution checkpoints to provide dense process-level supervision for reinforcement learning.
- **Empirical Effectiveness:** Extensive evaluations demonstrate that both SFT and GRPO models trained on our synthesized data achieve significant, consistent gains across diverse mathemati-

cal, general reasoning, and fine-grained symbolic benchmarks.

2 Related Work

Synthetic reasoning data and verifiable-reward training. Recent work studies synthetic reasoning data through augmentation, large-scale curation, or problem generation, primarily in mathematical domains (Yu et al., 2024a; Luo et al., 2023; LI et al., 2024; Toshniwal et al., 2024; Ding et al., 2024; Muennighoff et al., 2025; Ye et al., 2025). Related efforts also construct verifiable logic and puzzle tasks with rule-based generators (Liu et al., 2025; Chen et al., 2026; Ding et al., 2026b). In parallel, reinforcement learning from verifiable rewards has become a promising recipe for reasoning, ranging from pure outcome-based RL to process supervision with reward models, human annotations, Monte Carlo estimation, or search (Guo et al., 2025; Abel, 2019; Hu et al., 2026; Zhang et al., 2025b; Zheng et al., 2024a; Wang et al., 2024a; Ding et al., 2026a; Liao et al., 2026a). However, prior work typically relies on mathematical verifiers or separately trained process reward models, whereas we use programs as executable oracles that provide both gold outputs and verifiable intermediate signals.

Code and reasoning. A separate line of work uses code to support reasoning (Yang et al., 2025b), either as an inference-time tool (Gao et al., 2023; Chen et al., 2022b; Lv et al., 2024), as an evaluation substrate in programming benchmarks (Chen et al., 2021; Austin et al., 2021), or as a source of input–output reasoning tasks (Gu et al., 2024; Bao et al., 2025). Recent works also use algorithmic problems, execution supervision, or synthetic verifiable data to improve reasoning (Bao et al., 2025; Chen et al., 2025; Jung et al., 2025; Liu et al., 2025; Chen et al., 2026). **In contrast**, we do not use algorithmic problems in their original or lightly transformed forms as supervision. Instead, we transplant their executable logic into natural narratives, reducing programming-specific biases while preserving verifiable reasoning structure.

Reasoning failures on some trivial tasks. Recent benchmarks show that LLMs remain weak on tasks that are trivial for deterministic programs and human (Malek et al., 2025), including basic string manipulation, character-level reasoning, and text-based maze navigation (Wang et al., 2025; Uzan and Pinter, 2026; Dao and Vu, 2025). Several studies attribute these failures partly to tokenization-induced mismatches between symbolic units and model processing units (Zhang et al., 2024; Singh and Strouse, 2024; Fu et al., 2024). **This gap motivates our setting**: we target such code-trivial but reasoning-fragile tasks and improve them with program-grounded supervision.

3 Methodology

3.1 Framework Overview and Notations

We formally define the components of our proposed framework, `MIMIC`. Let $\mathbb{P}$ denote the domain of raw competitive programming problems. Each problem $p \in \mathbb{P}$ is parameterized as a tuple $p = (\mathcal{S}_p, \mathcal{I}_p, \mathcal{O}_p, f_p)$, where $\mathcal{S}_p \in \Sigma$ represents the raw natural language problem statement within the universal textual space Σ, $\mathcal{I}_p$ is the valid input space, $\mathcal{O}_p$ is the target output space, and $f_p : \mathcal{I}_p \to \mathcal{O}_p \in \mathbb{F}$ denotes the accepted code (AC) representing the ground-truth deterministic execution mapping within the universal program space $\mathbb{F}$. To transform the raw algorithmic tuple p into a robust, hallucination-mitigated set of natural-language reasoning pairs, `MIMIC` executes a sequential, four-stage pipeline (see Figure 2): (1) Problem Collection and Mathematical Kernel Extraction, (2) Problem Synthesis via Narrative Fusion, (3) Code-Guided Test Synthesis and Narrative Input Fusion, and (4) Trace-Grounded Chain-of-Thought (CoT) Synthesis.

Structurally, this sequential pipeline directly resolves the three core challenges identified in Section 1. To prevent shortcut memorization, Stages 1 and 2 project the algorithmic kernel away from recognizable programmatic artifacts and fuse it into an external, real-world narrative corpus, effectively decorrelating the underlying mathematical logic from its surface phrasing. To ensure generalizable coverage, Stage 3 bypasses naive random sampling by introspecting the control-flow boundaries of the executable code f_p to strategically generate targeted adversarial and guided-random test inputs. Finally, to eliminate hallucinations, Stage 4 dynamically instruments the accepted code to expose deterministic intermediate execution states, ensuring that every generated quantitative deduction is verifiably grounded in computational reality rather than autoregressive prior distributions.

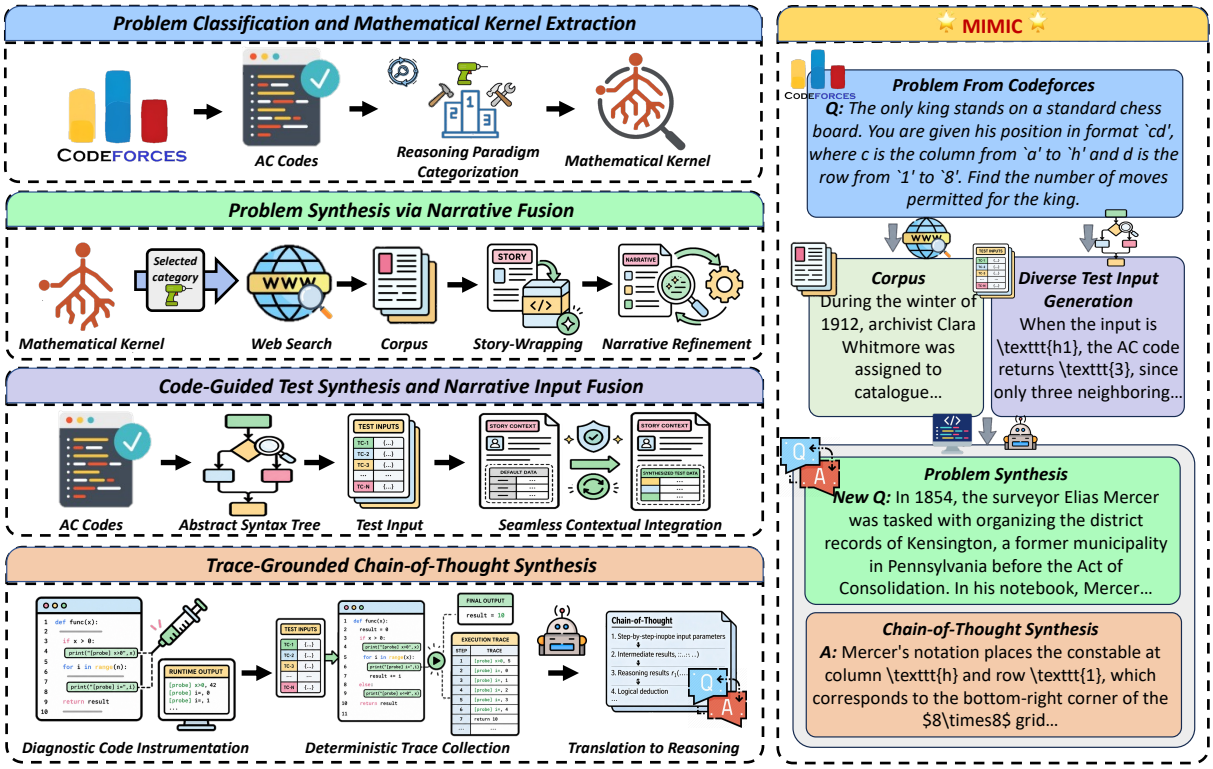


Figure 2: Overview of the MIMIC framework. It transforms raw algorithmic problems into verifiable reasoning data through a four-stage pipeline: (1) Problem Classification and Mathematical Kernel Extraction, (2) Problem Synthesis via Narrative Fusion, (3) Code-Guided Test Synthesis and Narrative Input Fusion, and (4) Trace-Grounded Chain-of-Thought (CoT) Synthesis.

3.2 Stage 1: Problem Classification and Mathematical Kernel Extraction

Reasoning Paradigm Categorization. In the initial step, MIMIC standardizes the raw problem $p \in \mathbb{P}$ by mapping it into a structured conceptual space. Let $\mathbb{T} = \{\tau_1, \tau_2, \ldots, \tau_k\}$ denote a predefined taxonomy of reasoning paradigms (e.g., precise mathematical computation, strict state simulation). MIMIC applies a classification operator $\Phi_{\text{cls}} : \mathbb{P} \to \mathbb{T}$ to assign an optimal reasoning category $t_p = \Phi_{\text{cls}}(p) \in \mathbb{T}$ to each problem, ensuring type-aware processing in all subsequent operations.

Algorithmic Kernel Isolation. Concurrently, to eliminate contest-specific narrative framing, MIMIC isolates the procedural semantics from the raw problem statement $\mathcal{S}_p$. We utilize a semantic refinement mapping $\Phi_{\text{ref}} : \Sigma \to \mathbb{K}$, where $\mathbb{K}$ is defined as the abstract space of concise mathematical kernels. The extracted kernel $K_p = \Phi_{\text{ref}}(\mathcal{S}_p)$ strictly preserves the algorithmic constraints, boundary conditions, and objective function operationalized by f_p, whilst discarding extraneous textual metadata. Stage 1 thus formally projects the raw tuple p into a normalized semantic representation (K_p, t_p, f_p). Further details on the taxonomy $\mathbb{T}$ and the extraction procedure are provided in Appendix A.

3.3 Stage 2: Problem Synthesis via Narrative Fusion

Type-Aware Corpus Sampling. To synthesize problems that authentically emulate real-world human-written exams without triggering memorized code templates, MIMIC relies on an external, large-scale educational corpus $\mathbb{C}$. To maintain semantic coherence, MIMIC partitions $\mathbb{C}$ into type-compatible subsets $\mathbb{C}_\tau \subset \mathbb{C}$ for each $\tau \in \mathbb{T}$. Given the problem type t_p, MIMIC samples an appropriate context passage $c \sim \mathbb{C}_{t_p}$.

Story-Wrapping Generation. Next, MIMIC fuses the extracted kernel K_p with the sampled context c. We formulate a story-wrapping generator $\Psi_{\text{wrap}} : \mathbb{K} \times \mathbb{C} \to \mathbb{Q}$, where $\mathbb{Q}$ is the space of natural-language exam questions. This generator integrates the abstract mathematical logic of K_p into the narrative environment of c, producing a base story-wrapped question $\tilde{Q}_{p,c} = \Psi_{\text{wrap}}(K_p, c)$. This initial projection inherently embeds a default valid input $\tilde{x} \in \mathcal{I}_p$ implicitly within its prose.

Narrative Refinement Optimization. MIMIC applies a narrative smoothing function $\Psi_{\text{smooth}} : \mathbb{Q} \to \mathbb{Q}$ to improve continuity between the retrieved context and problem formulation, producing the refined question template $Q^*_{p,c} = \Psi_{\text{smooth}}(\widetilde{Q}_{p,c})$. Details are provided in Appendix A.2.

3.4 Stage 3: Code-Guided Test Synthesis and Narrative Input Fusion

Control-Flow Introspection. Relying solely on the embedded default input $\tilde{x}$ yields insufficient coverage over the logical domain. To ensure robust generalizability, MIMIC systematically analyzes the deterministic algorithmic structure of f_p. Let $\mathcal{D}(f_p)$ denote the set of salient control-flow decision points (e.g., branch conditions, loop terminations, and variable configurations) identified within the abstract syntax tree of f_p.

Targeted Input Synthesis. Using these decision points, MIMIC defines an input synthesis operator $\Gamma_{\text{test}} : \mathcal{D}(f_p) \to 2^{\mathcal{I}_p}$. It systematically generates a diverse set of test configurations $\mathcal{X}_p = \{x_1, x_2, \ldots, x_N\} \subset \mathcal{I}_p$, designed to stress distinct execution paths through a mixture of adversarial boundary cases and guided-random inputs.

Seamless Contextual Integration. For each generated input $x_i \in \mathcal{X}_p$, MIMIC must update the associated narrative without violating the underlying story context. We formulate a narrative input fusion function $\Omega_{\text{fuse}} : \mathbb{Q} \times \mathcal{I}_p \times \mathcal{I}_p \to \mathbb{Q}$, which structurally replaces the original default parameters $\tilde{x}$ within $Q^*_{p,c}$ with the newly synthesized input x_i. This mapping yields an instantiated, input-specific natural language question $Q^{(i)}_{p,c} = \Omega_{\text{fuse}}(Q^*_{p,c}, \tilde{x}, x_i)$. The comprehensive methodology for decision-point extraction and narrative input fusion is detailed in Appendix A.3.

3.5 Stage 4: Trace-Grounded Chain-of-Thought Synthesis

Diagnostic Code Instrumentation. In the final stage, MIMIC aims to generate hallucination-mitigated reasoning trajectories. Rather than relying on autoregressive prior distributions to hallucinate intermediate steps, MIMIC structurally instruments the accepted code f_p. Let $\Lambda_{\text{inst}} : \mathbb{F} \to \mathbb{F}^*$ be a read-only instrumentation mapping that injects sparse diagnostic probes at semantically meaningful breakpoints within f_p, yielding the instrumented program $f^*_p = \Lambda_{\text{inst}}(f_p)$.

Deterministic Trace Collection. For each instantiated input $x_i \in \mathcal{X}_p$, executing the instrumented program $f^*_p(x_i)$ deterministically produces both the ground-truth output $y_i \in \mathcal{O}_p$ (where $y_i = f_p(x_i)$) and a corresponding sequence of intermediate execution states $E_i = (s_1, s_2, \ldots, s_m) \in \mathbb{E}$, where $\mathbb{E}$ is the sequential space of execution traces.

Procedural Translation to Reasoning. Finally, MIMIC utilizes a reasoning synthesis function $\Theta_{\text{CoT}} : \mathbb{Q} \times \mathbb{E} \times \mathcal{O}_p \to \mathbb{R}$ which translates the procedural scaffold E_i into a natural-language chain-of-thought $R^{(i)}_{p,c} \in \mathbb{R}$. The resulting reasoning sequence $R^{(i)}_{p,c}$ bridges the premise $Q^{(i)}_{p,c}$ and the conclusion y_i, ensuring that every deductive step is grounded in computational reality. The final training instance is the verified reasoning pair $(Q^{(i)}_{p,c}, R^{(i)}_{p,c})$. Details on trace collection and CoT generation are provided in Appendix A.4.

4 Experiment

In this section, we conduct extensive experiments to answer the following research questions (RQs):

- **RQ1:** Can our method enhance LLMs' reasoning capabilities, and does it generalize to mathematical and other general reasoning tasks?
- **RQ2:** Can our method effectively solve fine-grained, rule-based deterministic problems (e.g., character-level perception or spatial reasoning)?
- **RQ3:** Does our data synthesis pipeline genuinely elicit reasoning rather than triggering superficial pattern matching or memorized recall?
- **RQ4:** Is our code-instrumented process reward effective for reinforcing reasoning?

4.1 Experimental Setup

We briefly summarize our experimental setup. For base models, we use Qwen3-4B, Qwen3-8B, and DeepSeek-R1-Distill-8B. Our training incorporates both Supervised Fine-Tuning (SFT) and Group Relative Policy Optimization (GRPO). The synthesized MIMIC dataset is utilized alongside mathematical datasets to study composability. We evaluate across general reasoning (e.g., ARC, BBH, GPQA, MMLU, DROP), mathematical benchmarks (GSM8K, MATH, AIME), coding benchmarks (HumanEval, MBPP), and fine-grained tasks (CharBench, StringBench, MazeBench). Comprehensive details regarding the datasets, training hyperparameters, algorithms, and evaluation metrics

Model	Setting	General Reasoning							Math					Code			
		ARC	BBH	MIMIC	GPQA	MMLU	DROP	**Avg**	GSM8K	GSM+	MATH	AIME	**Avg**	HEval	CruxE	MBPP	**Avg**
(A) Main Results: Baseline → Ours (SFT) / Ours (GRPO)																	
Qwen3-4B	Base	79.6	76.0	40.6	40.1	66.3	50.5	58.9	89.8	73.2	78.3	15.6	64.2	31.7	69.3	39.7	46.9
	Ours (SFT)	**85.9**	77.0	43.5	40.4	67.3	55.5	61.6 ↑2.7	**91.0**	74.9	77.6	16.7	65.0 ↑0.8	**32.1**	70.8	38.8	47.2 ↑0.3
	Ours (GRPO)	80.6	**77.9**	**51.3**	**40.8**	**67.5**	**58.3**	**62.7** ↑3.8	**91.0**	**76.4**	**80.3**	**28.3**	**69.0** ↑4.8	31.7	**72.7**	**43.6**	**49.3** ↑2.4
Qwen3-8B	Base	79.8	78.8	41.2	**44.4**	70.5	46.6	60.2	90.6	73.7	79.2	23.3	66.7	22.6	70.6	40.5	44.6
	Ours (SFT)	**85.3**	78.5	44.5	42.0	69.5	50.4	61.7 ↑1.5	90.1	74.5	80.2	**31.7**	69.1 ↑2.4	**27.4**	72.8	34.2	44.8 ↑0.2
	Ours (GRPO)	81.2	**81.7**	**52.2**	42.6	**72.2**	**53.1**	**63.8** ↑3.6	**91.7**	**78.1**	**81.0**	26.7	**69.4** ↑2.7	23.8	**76.3**	**44.0**	**48.0** ↑3.4
DeepSeek-R1 -Distill-8B	Base	83.0	69.1	38.3	24.0	52.8	18.1	47.6	82.5	60.3	58.4	16.7	54.5	22.6	44.8	17.5	28.3
	Ours (SFT)	**86.1**	**75.7**	40.8	25.2	55.8	**23.6**	**51.2** ↑3.6	**88.5**	**70.4**	66.2	**26.7**	63.0 ↑8.5	23.9	38.9	**24.8**	29.2 ↑0.9
	Ours (GRPO)	83.5	68.5	**43.5**	**27.7**	**59.3**	19.1	50.2 ↑2.6	88.4	66.7	**78.2**	23.3	**64.2** ↑9.7	**26.8**	**48.3**	21.8	**32.3** ↑4.0
(B) Synthesis Ablation: Ours (Contextualized) vs. Raw Codeforces																	
Qwen3-4B	Ours (SFT)	**85.9**	**77.0**	**43.5**	**40.4**	**67.3**	**55.5**	**61.6**	**91.0**	**74.9**	**77.6**	**16.7**	**65.0**	32.1	**70.8**	**35.8**	**46.2**
	Codeforces (SFT)	85.2	43.9	28.9	25.4	44.8	42.6	45.1 ↓16.5	71.5	54.9	40.4	6.7	43.4 ↓21.6	**34.2**	65.6	34.6	44.8 ↓1.4
	Ours (GRPO)	**80.6**	**77.9**	**51.3**	**40.8**	**67.5**	**58.3**	**62.7**	**91.0**	**76.4**	**80.3**	**28.3**	**69.0**	**31.7**	**72.7**	**43.6**	**49.3**
	Codeforces (GRPO)	80.1	75.1	45.1	32.4	65.1	41.2	56.5 ↓6.2	89.4	75.6	75.5	20.0	65.1 ↓3.9	25.6	66.8	33.9	42.1 ↓7.2
Qwen3-8B	Ours (SFT)	**85.3**	**78.5**	**44.5**	**42.0**	**69.5**	**50.4**	**61.7**	**91.6**	**74.5**	**80.2**	**32.7**	**67.0**	27.4	72.8	34.2	44.8
	Codeforces (SFT)	81.7	44.1	28.7	24.7	39.4	31.3	41.7 ↓20.0	74.2	57.1	41.9	6.7	45.0 ↓22.0	**32.3**	**72.8**	**35.8**	**47.0** ↑2.2
	Ours (GRPO)	**81.2**	**81.7**	**52.2**	**42.6**	**72.2**	**53.1**	**63.8**	**91.7**	**78.1**	81.0	**26.7**	**69.4**	**23.8**	**76.3**	**44.0**	**48.0**
	Codeforces (GRPO)	79.4	81.1	45.8	40.2	67.3	36.7	58.4 ↓5.4	**91.7**	76.5	**81.2**	20.0	67.3 ↓2.1	21.3	76.0	42.4	46.6 ↓1.4
DeepSeek-R1 -Distill-8B	Ours (SFT)	**86.1**	**75.7**	**40.8**	**25.2**	**55.8**	**23.6**	**51.2**	**88.5**	**70.4**	**66.2**	**26.7**	**63.0**	23.9	**38.9**	**24.8**	29.2
	Codeforces (SFT)	83.8	43.8	36.6	21.5	43.0	22.7	41.9 ↓9.3	62.0	49.1	55.4	23.3	47.5 ↓15.5	**27.4**	35.1	14.8	25.8 ↓3.4
	Ours (GRPO)	**83.5**	**68.5**	**43.5**	**27.7**	**59.3**	**19.1**	**50.2**	**88.4**	66.7	**78.2**	**23.3**	**64.2**	**26.8**	**48.3**	**21.8**	**32.3**
	Codeforces (GRPO)	80.8	61.2	41.1	24.3	57.8	18.7	47.3 ↓2.9	74.8	**68.1**	74.7	15.0	58.2 ↓6.0	**26.8**	38.4	16.7	27.3 ↓5.0
(C) Cross-Domain Mixing: Composability with Mathematical Data																	
Qwen3-4B	Math (SFT)	85.2	**77.6**	39.4	38.3	66.3	51.3	59.7	90.2	**75.3**	75.9	**16.7**	64.5	**34.2**	69.5	31.1	44.9
	Ours (SFT)	**85.9**	77.0	**43.5**	**40.4**	**67.3**	**55.5**	**61.6** ↑1.9	**91.0**	74.9	**77.6**	**16.7**	**65.0** ↑0.5	32.1	**70.8**	**35.8**	**46.2** ↑1.3
	$\mathcal{D}_{\text{math}}$ (GRPO)	80.2	77.8	41.4	38.6	**68.7**	39.9	57.8	91.7	76.6	**81.7**	23.3	68.3	27.4	70.5	39.3	45.8
	Ours (GRPO)	**80.6**	**77.9**	**51.3**	**40.8**	67.5	**58.3**	**62.7** ↑4.9	91.0	76.4	80.3	28.3	69.0 ↑0.7	**31.7**	72.7	**43.6**	**49.3** ↑3.5
	$\mathcal{D}_{\text{mixed}}$ (GRPO)	79.3	77.6	51.0	39.1	67.8	41.6	59.4 ↑1.6	**93.2**	**76.7**	78.6	**30.0**	**69.6** ↑1.3	27.4	**75.7**	39.9	47.7 ↑1.9
Qwen3-8B	Math (SFT)	84.0	**78.5**	39.9	40.8	69.0	48.8	60.2	**91.7**	**75.2**	**82.0**	23.3	68.1	20.1	66.5	**37.0**	41.2
	Ours (SFT)	**85.3**	**78.5**	**44.5**	**42.0**	**69.5**	**50.4**	**61.7** ↑1.5	91.6	74.5	80.2	**32.7**	**69.8** ↑1.7	**27.4**	**72.8**	34.2	**44.8** ↑3.6
	$\mathcal{D}_{\text{math}}$ (GRPO)	78.6	81.2	42.0	**44.4**	68.4	36.2	58.5	91.0	**78.2**	**83.7**	26.7	69.9	19.5	73.5	45.9	46.3
	Ours (GRPO)	81.2	**81.7**	**52.2**	42.6	**72.2**	**53.1**	**63.8** ↑5.3	91.7	78.1	81.0	26.7	69.4 ↑0.5	**23.8**	76.3	44.0	48.0 ↑1.7
	$\mathcal{D}_{\text{mixed}}$ (GRPO)	**81.9**	80.7	51.0	43.9	69.1	37.3	60.6 ↑2.1	**92.1**	77.7	81.6	**30.0**	**70.3** ↑0.4	23.2	**77.2**	**49.4**	**49.9** ↑3.6
DeepSeek-R1 -Distill-8B	Math (SFT)	85.1	**75.8**	36.3	24.0	**57.4**	**26.5**	50.9	**89.6**	**71.2**	**67.2**	3.3	57.8	**26.3**	32.8	**25.3**	28.1
	Ours (SFT)	**86.1**	75.7	**40.8**	**25.2**	55.8	23.6	**51.2** ↑0.3	88.5	70.4	66.2	**26.7**	**63.0** ↑5.2	23.9	**38.9**	24.8	**29.2** ↑1.1
	$\mathcal{D}_{\text{math}}$ (GRPO)	81.1	**75.0**	38.4	23.9	58.9	**20.8**	49.7	91.4	**74.3**	73.4	20.0	64.8	22.4	45.4	11.6	26.5
	Ours (GRPO)	83.5	68.5	**43.5**	**27.7**	59.3	19.1	50.2 ↑0.5	88.4	66.7	78.2	**23.3**	64.2 ↑0.6	**26.8**	**48.3**	**21.8**	**32.3** ↑5.8
	$\mathcal{D}_{\text{mixed}}$ (GRPO)	**84.2**	70.6	42.4	26.6	**59.7**	19.4	**50.5** ↑0.8	**91.7**	73.1	**80.7**	20.0	**66.4** ↑1.6	21.8	46.3	11.9	26.7 ↑0.2
(D) Robustness across RL Algorithms (Qwen3-8B)																	
Qwen3-8B	Base	79.8	78.8	41.2	**44.4**	70.5	46.6	60.2	90.6	73.7	79.2	23.3	66.7	22.6	70.6	40.5	44.6
Qwen3-8B	GRPO	81.2	**81.7**	52.2	42.6	72.2	**53.1**	**63.8** ↑3.6	91.7	78.1	81.0	26.7	69.4 ↑2.7	**23.8**	76.3	**44.0**	**48.0** ↑3.4
	GPG	81.0	81.4	52.1	41.5	**72.6**	53.0	63.6 ↑3.4	92.0	77.5	81.3	26.7	69.4 ↑2.7	21.3	74.4	41.2	45.7 ↑1.1
	RLOO	79.3	81.4	**52.8**	41.1	71.2	52.4	63.0 ↑2.8	**93.6**	**78.2**	**81.7**	**31.7**	**71.3** ↑4.6	21.3	**77.4**	42.8	47.2 ↑2.6
	Reinforce++	**81.4**	80.7	46.8	43.1	71.3	52.7	62.7 ↑2.5	91.4	75.1	80.8	30.0	69.3 ↑2.6	22.8	70.9	40.1	44.6 ↑0

Table 1: **Main Results and Ablation Studies.** (A) Main results comparing base models with our MIMIC data via SFT and GRPO. (B) Data source ablation: our contextualized synthesis vs. raw Codeforces problems. (C) Cross-domain mixing with mathematical/benchmark data. (D) RL algorithm robustness. **Avg** = category mean; subscripts show absolute change vs. the reference row. Best per group in **bold**.

are provided in Appendix B and C.

4.2 RQ1: Generalization of Reasoning Capabilities

To answer **RQ1**, we present a comprehensive evaluation across general reasoning, mathematical, and coding benchmarks in Table 1. The results explicitly confirm that our method significantly enhances reasoning capabilities and generalizes profoundly across domains.

Main Results (Panel A): Across all three base model families, applying our MIMIC data through SFT and GRPO yields substantial performance leaps. For instance, on Qwen3-4B, GRPO lifts the general reasoning average by 3.8 points and the mathematical reasoning average by 4.8 points, highlighted by a staggering improvement on AIME (15.6 → 28.3). Notably, the DeepSeek-R1-Distill-8B model experiences the most dramatic gains in math (+9.7 average), proving that our methodology is highly effective even on already distilled reasoning models. Furthermore, contrary to the typical alignment tax where reasoning improvements degrade coding skills, our models maintain or improve on coding benchmarks (e.g., Qwen3-8B MBPP improves from 40.5 to 44.0).

Synthesis Ablation (Panel B): To isolate the source of improvement, we compared our contextualized synthesis with training on raw Codeforces problems, where the same test cases were incorporated using the same input-fuse procedure. The contrast is stark: directly fine-tuning or applying

RL on raw Codeforces data heavily harms general reasoning. For example, Qwen3-4B trained on raw Codeforces (SFT) suffers a 16.5-point drop on general reasoning and a 21.6-point drop on math. This confirms that merely exposing models to code data induces overfitting to syntax at the expense of general logic, whereas our narrative fusion effectively distills algorithmic logic into a universally applicable reasoning format.

Composability and RL Robustness (Panels C & D): Panel (C) highlights the exceptional composability of our data. When mixing `MIMIC` with mathematical data ($\mathcal{D}_{\text{mixed}}$), Qwen3-4B achieves the best of both worlds, setting peak scores on GSM8K (93.2) and AIME (30.0) without compromising general reasoning. Finally, Panel (D) explores algorithmic robustness. While GRPO, GPG, RLOO, and Reinforce++ all yield solid improvements over the base model, the gains remain largely consistent across different RL optimization strategies. This indicates that the core driver of our success stems from the high-quality, hallucination-mitigated reasoning trajectories synthesized by our pipeline, rather than reliance on any single specific RL algorithm.

Comparison with recent baselines: Following reviewer suggestions, we additionally compare MIMIC against several closely related methods, including TeaR, CoE, ExecGrounded, SynLogic, and Enigmata. As shown in Table 2, MIMIC consistently outperforms these baselines across the majority of evaluation settings. We believe this result further supports our central claim: preserving executable logic alone is insufficient if the supervision still retains strong programming-specific artifacts, whereas our narrative fusion formulation yields more transferable reasoning signals.

4.3 RQ2: Solving Fine-grained, Rule-based Deterministic Problems

Regarding **RQ2**, we investigate whether our models can overcome the notorious vulnerabilities LLMs face with tokenization artifacts and fine-grained, deterministic problems. To assess this, we tested the models on MazeBench (spatial state tracking), StringBench, and CharBench (character-level perception) in Table 3[1]. It is crucial to note that *none of these specific benchmarks or their exact formats were included in our training data*.

The results demonstrate profound, zero-shot generalizations to deterministic rules. Qwen3-8B with GRPO demonstrates remarkable improvements across benchmarks, jumping to 40.0% (+82% relative gain) on MazeBench, rising to 39.5% on StringBench to rival larger proprietary models, and achieving a near-perfect 99.2% accuracy on CharBench. By grounding the training in explicit execution traces and step-by-step state tracking, `MIMIC` fundamentally alters how the model perceives inputs. Instead of relying on blurry semantic approximations caused by BPE tokenization, the model learns to internalize strict, deterministic state perception, allowing it to robustly process and manipulate token structures.[2]

4.4 RQ3: Reasoning vs. Pattern Matching

After discussing large-scale training results, we address **RQ3** by examining whether our synthesized data encourages reasoning rather than recall of memorized code templates. We conduct a preliminary study on QWEN3-32B (context length 16384) under four problem presentation formats: (1) **original**, the raw Codeforces statement; (2) **refined**, a minimal version with the narrative background removed; (3) **fused** (ours), where the task is embedded into a newly constructed natural-language context; and (4) **synthesized**, where the model directly generates the final test item without constraint. We then evaluate how these formats affect model behavior and perceived difficulty.

Table 4 provides a clear empirical answer to **RQ3** by isolating the impact of problem presentation. When processing the **original** Codeforces statements, the model exhibits a strong bias toward pattern matching rather than genuine reasoning, attempting to write raw code snippets in 38.7% of the cases. This "recall shortcut" drastically harms performance (18.4% accuracy with code vs. 27.2% without), as the model wastes context capacity reconstructing memorized competitive-programming templates rather than analyzing the specific logic of the given test case. Merely stripping the narrative (the **refined** format) only partially mitigates this issue, leaving a high code generation rate of 29.9%.

Crucially, our **fused** format fundamentally changes the model's behavior. By embedding the underlying algorithmic structure into novel, everyday natural-language scenarios, it removes the semantic cues that trigger template recall. As a result,

[1] Because Deepseek-R1-Distilled-8B only has think mode, the "directly output answer" constraint of CharBench does not apply to it; we only report performance with cot.

[2] To illustrate this, we provide case studies in Appendix I

Model	Setting	General Reasoning							Math					Code			
		ARC	BBH	MIMIC	GPQA	MMLU	DROP	**Avg**	GSM8K	GSM+	MATH	AIME	**Avg**	HEval	CruxE	MBPP	**Avg**
Comparison with Closely Related Baselines																	
(A) SFT																	
Qwen3-4B	Ours (SFT)	**85.9**	**77.0**	**43.5**	**40.4**	**67.3**	**55.5**	**61.6**	**91.0**	**74.9**	**77.6**	**16.7**	**65.0**	32.1	**70.8**	35.8	**46.2**
	CoE (SFT)	85.2	43.9	28.9	25.4	44.8	42.6	45.1 ↓16.5	71.5	54.9	40.4	6.7	43.4 ↓21.6	34.2	65.6	34.6	44.8 ↓1.4
	ExecGrounded (SFT)	75.4	66.9	39.2	35.0	40.8	26.1	47.2 ↓14.4	77.6	63.4	74.4	**16.7**	58.0 ↓7.0	23.8	63.0	**45.5**	44.1 ↓2.1
	Enigmata (SFT)	72.0	49.1	21.5	34.6	30.1	37.1	40.7 ↓20.9	60.9	53.8	69.2	13.3	49.3 ↓15.7	**35.4**	41.9	24.9	34.1 ↓12.1
DeepSeek-R1 -Distill-8B	Ours (SFT)	**86.1**	**75.7**	**40.8**	25.2	**55.8**	**23.6**	**51.2**	**88.5**	**70.4**	66.2	**26.7**	**63.0**	23.9	**38.9**	24.8	**29.2**
	CoE (SFT)	83.8	43.8	36.6	21.5	43.0	22.7	41.9 ↓9.3	62.0	49.1	55.4	23.3	47.5 ↓15.5	**27.4**	35.1	14.8	25.8 ↓3.4
	ExecGrounded (SFT)	76.3	59.3	38.9	25.5	51.3	10.0	43.6 ↓7.6	66.5	49.1	**77.1**	23.3	54.0 ↓9.0	12.8	35.6	20.6	23.0 ↓6.2
	Enigmata (SFT)	75.6	37.7	18.7	**28.7**	41.1	10.0	35.3 ↓15.9	61.1	40.7	62.3	20.0	46.0 ↓17.0	21.3	34.9	**26.2**	27.5 ↓1.7
(B) GRPO																	
Qwen3-4B	Ours (GRPO)	**80.6**	**77.9**	**51.3**	40.8	**67.5**	**58.3**	**62.7**	**91.0**	**76.4**	80.3	**28.3**	**69.0**	**31.7**	**72.7**	**43.6**	**49.3**
	TeaR (GRPO)	80.1	75.1	45.1	32.4	65.1	41.2	56.5 ↓6.2	89.4	75.6	75.5	20.0	65.1 ↓3.9	25.6	66.8	33.9	42.1 ↓7.2
	SynLogic (GRPO)	77.1	76.2	41.0	39.5	62.3	41.7	56.3 ↓6.4	90.2	69.0	**81.3**	23.3	66.0 ↓3.0	26.8	64.2	37.0	42.7 ↓6.6
	Enigmata (GRPO)	76.8	75.6	41.0	**41.5**	62.4	42.1	56.6 ↓6.1	89.0	67.9	80.6	23.3	65.2 ↓3.8	29.9	68.8	36.2	45.0 ↓4.3
DeepSeek-R1 -Distill-8B	Ours (GRPO)	**83.5**	**68.5**	**43.5**	27.7	**59.3**	**19.1**	**50.3**	**88.4**	66.7	78.2	**23.3**	**64.2**	**26.8**	**48.3**	**21.8**	**32.3**
	TeaR (GRPO)	80.8	61.2	41.1	24.3	57.8	18.7	47.3 ↓3.0	74.8	**68.1**	74.7	15.0	58.1 ↓6.1	**26.8**	38.4	16.7	27.3 ↓5.0
	SynLogic (GRPO)	76.1	62.4	40.3	27.9	44.6	14.3	44.3 ↓6.0	83.9	58.7	76.2	20.0	59.7 ↓4.5	20.7	33.4	12.8	22.3 ↓10.0
	Enigmata (GRPO)	75.4	56.5	38.7	**30.7**	47.8	12.3	43.6 ↓6.7	76.0	57.5	**78.3**	**23.3**	58.8 ↓5.4	22.0	37.6	16.0	25.2 ↓7.1

Table 2: Comparison with closely related baselines. We compare MIMIC against representative recent methods under both SFT and GRPO. **Avg** = category mean; subscripts show absolute change vs. the reference row. Best per group in **bold**.

Model	Method	Maze	String	Char	Char+CoT
Qwen3-4B	Base	16.0	13.6	37.9	97.1
	+SFT	18.0	22.8	**40.4**	97.8
	+GRPO	**22.0**	**26.8**	39.1	**98.2**
Qwen3-8B	Base	22.0	26.2	38.6	98.0
	+SFT	22.0	37.5	39.3	98.6
	+GRPO	**40.0**	**39.5**	**41.3**	**99.2**
DeepSeek-R1 -Distill-8B	Base	10.0	14.0	—	89.2
	+SFT	11.0	**22.2**	—	91.2
	+GRPO	**14.0**	20.0	—	**94.7**

Table 3: **Character-level and spatial reasoning benchmarks.** We evaluate on benchmarks targeting fine-grained perception and rules (e.g., comparing string patterns like 9.11 and 9.9): CharBench, StringBench, and MazeBench. Best per model family in **bold**.

the code rate drops sharply to 0.7%, while reasoning accuracy rises to 42.2% (vs. 23.8% for the original format), indicating that the model is driven toward step-by-step deduction rather than shallow pattern matching. Although the fully **synthesized** format also yields a low code rate, its lack of constraint causes the generated test cases to be overly simple and stereotyped (accuracy 61.8%), failing to capture the adversarial hardness of rigorous benchmarks. Therefore, our fused format strikes a better balance: it effectively eliminates superficial memorization while preserving the intricate deductive complexity required for robust reasoning[3].

[3]More analyses, case studies, and results are provided in Appendix D.

Format	Acc. (%)	Code Rate (%)	Acc. (%) w/ Code	Acc. (%) w/o Code
Original	23.8	38.7	18.4	27.2
Refined	34.4	29.9	30.6	36.0
Fused (Ours)	42.2	0.7	15.0	42.4
Synthesized	61.8	0.8	41.3	62.0

Table 4: Preliminary comparison of different problem presentation formats on QWEN3-8B. "Code Rate" denotes the proportion of cases where the model attempted to write code during reasoning.

4.5 RQ4: Effectiveness of the Process Reward

Finally, to answer **RQ4**, we ablate the effectiveness of our reward modeling strategy under the GRPO framework. Table 5 compares our Code-Instrumented Reward (CIR) against a sparse Binary outcome reward, as well as two state-of-the-art, learned Process Reward Models (PRM800K and Math-Shepherd). While a simple Binary reward provides a solid baseline improvement (+3.6 on General Avg, +2.7 on Math Avg for Qwen3-8B), it lacks the granularity to effectively penalize intermediate logical fallacies. On the other hand, injecting external learned PRMs yields surprisingly meager gains. For instance, PRM800K only improves the General Avg by 1.2 points, and Math-Shepherd actually degrades mathematical reasoning slightly relative to the base average (65.3 vs. 66.7). This underperformance stems from the fact that existing PRMs are heavily overfitted to their specific training distributions (typically step-by-step mathematical derivations) and actively struggle to reliably score the diverse, highly structured algorithmic reasoning trajectories generated by our pipeline.

Model	Reward	General Reasoning							Math					Code			
		ARC	BBH	MIMIC	GPQA	MMLU	DROP	**Avg**	GSM8K	GSM+	MATH	AIME	**Avg**	HEval	CruxE	MBPP	**Avg**
Qwen3-8B	Base	79.8	78.8	41.2	44.4	70.5	46.6	60.2	90.6	73.7	79.2	23.3	66.7	22.6	70.6	40.5	44.6
	Binary (GRPO)	81.2	**81.7**	52.2	42.6	**72.2**	**53.1**	63.8$_{\uparrow 3.6}$	91.7	**78.1**	81.0	26.7	69.4$_{\uparrow 2.7}$	23.8	**76.3**	44.0	**48.0**$_{\uparrow 3.4}$
	CIR (Ours)	**82.7**	80.6	**54.5**	**48.5**	71.6	53.5	**65.2**$_{\uparrow 5.0}$	**92.6**	78.0	**81.4**	**30.0**	**70.5**$_{\uparrow 3.8}$	**25.3**	76.2	40.9	47.5$_{\uparrow 2.9}$
	PRM800K	79.8	79.3	43.8	42.7	71.6	50.9	61.4$_{\uparrow 1.2}$	91.1	74.7	80.5	23.3	67.4$_{\uparrow 0.7}$	23.2	71.8	40.5	45.2$_{\uparrow 0.6}$
	Math-Shepherd	81.4	79.8	43.9	43.1	71.4	52.7	62.1$_{\uparrow 1.9}$	90.9	74.3	75.9	20.0	65.3$_{\downarrow 1.4}$	21.3	72.3	**44.4**	46.0$_{\uparrow 1.4}$

Table 5: **Reward Signal Ablation.** Comparing reward strategies under GRPO training: (1) **Binary** outcome reward; (2) **CIR** (Code-Instrumented Reward), our approach leveraging execution checkpoints; (3) **PRM800K**; (4) **Math-Shepherd**. Subscripts show absolute change vs. Base.

In stark contrast, our CIR methodology bypasses the need for an external, fallible neural network by leveraging deterministic execution checkpoints natively embedded in the synthesized code. This guarantees perfectly accurate, dense, and graded reward signals at each logical step. Consequently, CIR delivers the best results on both general reasoning and mathematical reasoning, surpassing binary reward and existing PRMs by +5.0 and +3.8 points over the baseline, respectively. By verifying algorithmic progress domain-agnostically through direct computation, CIR proves a highly effective and robust mechanism for reinforcing genuine reasoning in LLMs.

Extended Investigations and Key Insights. We further conduct additional experiments on MIMIC in the appendix. Appendix D presents analyses of the dataset distribution across categories. Appendix F examines changes in output patterns and token probabilities before and after training, providing insight into how training on our data affects model behavior. Appendix G reports training dynamics to illustrate how the model evolves during optimization on our data. Appendix H presents a task-type ablation study to analyze the contribution of each category during training.

5 Discussion and Conclusion

Beyond empirical gains, this work provokes critical reflections on LLM reasoning paradigms. First, it redefines code from a mere generation target into a rigorous *cognitive scaffold*. The structural determinism of programs forces LLMs to ground linguistic reasoning in verifiable logic, effectively bridging symbolic execution with semantic understanding to mitigate hallucination. Second, this paradigm democratizes Process Reward Models (PRMs). Because intermediate programmatic execution states inherently provide cost-free, high-fidelity step-level verification, MIMIC unlocks a scalable pathway for process supervision without relying on prohibitive human annotations or closed-source teacher models.

In conclusion, we propose MIMIC, a framework that translates executable code and state transitions into high-quality natural language reasoning trajectories. By compelling LLMs to imitate the deterministic logic of programs, we significantly enhance their mathematical and logical capabilities. Ultimately, shifting the "imitation game" toward program execution establishes a self-verifiable, sustainable data engine, paving the way for more interpretable and reliable AI reasoning.

Limitations

Reward signal granularity. Our Code-Instrumented Reward uses a simple step-counting formulation (k/K), treating all execution checkpoints equally. The primary goal of this design is not to propose an optimal reward model, but rather to demonstrate that the rich intermediate results naturally available from program execution can serve as effective training signals, even in their simplest form. The fact that this straightforward approach already outperforms established process reward models (Table 5) suggests that the deterministic, code-derived nature of the signal is itself the key ingredient.

Data source and scale. We currently use only Codeforces as the problem source, a subset of pre-training corpora for contextualization, and generate only ∼20 contextualized problems per seed problem, with all synthesis performed by a Qwen3.5-27B model rather than a frontier-scale system. While all of these (problem sources, corpus coverage, per-seed generation volume, and synthesizer model capacity) can be trivially scaled, the current configuration is already more than sufficient, comfortably supporting both SFT (∼114K samples) and RL (∼10K prompts) training with consistent improvements across all evaluated dimensions.

References

David Abel. 2019. simple_rl: Reproducible reinforcement learning in python. In *RML@ ICLR*. New Orleans, LA.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and 1 others. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*.

Keqin Bao, Nuo Chen, Xiaoyuan Li, Binyuan Hui, Bowen Yu, Fuli Feng, Xiangnan He, and Dayiheng Liu. 2025. Teaching llm to reason: Reinforcement learning from algorithmic problems without code. *Preprint*, arXiv:2507.07498.

Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. 2022a. Codet: Code generation with generated tests. *arXiv preprint arXiv:2207.10397*.

Jiangjie Chen, Qianyu He, Siyu Yuan, Aili Chen, Zhicheng Cai, Weinan Dai, Hongli Yu, Jiaze Chen, Xuefeng Li, Qiying Yu, and 1 others. 2026. Enigmata: Scaling logical reasoning in large language models with synthetic verifiable puzzles. *Advances in Neural Information Processing Systems*, 38:3613–3661.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, and 39 others. 2021. Evaluating large language models trained on code. *Preprint*, arXiv:2107.03374.

Nuo Chen, Zehua Li, Keqin Bao, Junyang Lin, and Dayiheng Liu. 2025. Chain of execution supervision promotes general reasoning in large language models. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2025, NeurIPS 2025, San Diego, CA, USA, December 2-7, 2025 / Mexico City, Mexico, November 30 - December 5, 2025*.

Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W Cohen. 2022b. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *arXiv preprint arXiv:2211.12588*.

Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2024. Teaching large language models to self-debug. In *International Conference on Learning Representations*, volume 2024, pages 8746–8825.

Tianzhe Chu, Yuexiang Zhai, Jihan Yang, Shengbang Tong, Saining Xie, Dale Schuurmans, Quoc V Le, Sergey Levine, and Yi Ma. 2025. Sft memorizes, rl generalizes: A comparative study of foundation model post-training. *arXiv preprint arXiv:2501.17161*.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv:1803.05457v1*.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, and 1 others. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.

Alan Dao and Dinh Bach Vu. 2025. Alphamaze: Enhancing large language models' spatial intelligence via grpo. *arXiv preprint arXiv:2502.14669*.

Hongxin Ding, Yue Fang, Runchuan Zhu, Xinke Jiang, Jinyang Zhang, Yongxin Xu, Weibin Liao, Xu Chu, Junfeng Zhao, and Yasha Wang. 2025. 3ds: Medical domain adaptation of llms via decomposed difficulty-based data selection. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 19473–19495.

Hongxin Ding, Baixiang Huang, Yue Fang, Weibin Liao, Xinke Jiang, Jinyang Zhang, Yinghao Zhu, Zheng Li, Liantao Ma, Junfeng Zhao, and 1 others. 2026a. Promed: Shapley information gain guided reinforcement learning for proactive medical llms. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 32481–32515.

Hongxin Ding, Baixiang Huang, Yue Fang, Weibin Liao, Zheng Li, Jinyang Zhang, Zhijing Wu, Junfeng Zhao, and Yasha Wang. 2026b. Evorubrics: Dynamic rubrics as rewards via adversarial co-evolution for llm reinforcement learning. *arXiv preprint arXiv:2606.23038*.

Yuyang Ding, Xinyu Shi, Xiaobo Liang, Juntao Li, Qiaoming Zhu, and Min Zhang. 2024. Unleashing reasoning capability of llms via scalable question synthesis from scratch. *https://arxiv.org/abs/2410.18693*.

Dheeru Dua, Yizhong Wang, Pradeep Dasigi, Gabriel Stanovsky, Sameer Singh, and Matt Gardner. 2019. Drop: A reading comprehension benchmark requiring discrete reasoning over paragraphs. *Preprint*, arXiv:1903.00161.

Nouha Dziri, Ximing Lu, Melanie Sclar, Xiang Lorraine Li, Liwei Jiang, Bill Yuchen Lin, Sean Welleck, Peter West, Chandra Bhagavatula, Ronan Le Bras, and 1 others. 2023. Faith and fate: Limits of transformers on compositionality. *Advances in neural information processing systems*, 36:70293–70332.

Tairan Fu, Raquel Ferrando, Javier Conde, Carlos Arriaga, and Pedro Reviriego. 2024. Why do large language models (llms) struggle to count letters? *arXiv preprint arXiv:2412.18626*.

Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, and 5 others. 2024. The language model evaluation harness.

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Pal: Program-aided language models. In *International conference on machine learning*, pages 10764–10799. PMLR.

Alex Gomez-Villa, Adrián Martín, Javier Vazquez-Corral, Marcelo Bertalmío, and Jesús Malo. 2022. On the synthesis of visual illusions using deep generative models. *Journal of Vision*, 22(8):2–2.

Alex Gu, Baptiste Rozière, Hugh Leather, Armando Solar-Lezama, Gabriel Synnaeve, and Sida I. Wang. 2024. Cruxeval: A benchmark for code reasoning, understanding and execution. *arXiv preprint arXiv:2401.03065*.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, and 1 others. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*.

Siyuan Guo, Aniket Didolkar, Nan Rosemary Ke, Anirudh Goyal, Ferenc Huszár, and Bernhard Schölkopf. 2024. Learning beyond pattern matching? assaying mathematical understanding in llms. *arXiv preprint arXiv:2405.15485*.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the math dataset. *NeurIPS*.

Jingcheng Hu, Yinmin Zhang, Qi Han, Daxin Jiang, Xiangyu Zhang, and Heung-Yeung Shum. 2026. Open-reasoner-zero: An open source approach to scaling up reinforcement learning on the base model. *Advances in Neural Information Processing Systems*, 38:162239–162262.

Hui Huang, Yancheng He, Hongli Zhou, Rui Zhang, Wei Liu, Weixun Wang, Jiaheng Liu, and Wenbo Su. 2026. Think-j: Learning to think for generative llm-as-a-judge. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 31158–31166.

Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, and 1 others. 2024. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*.

Denis Janiak, Jakub Binkowski, Albert Sawczyn, Bogdan Gabrys, Ravid Shwartz-Ziv, and Tomasz Jan Kajdanowicz. 2025. The illusion of progress: Re-evaluating hallucination detection in llms. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 34716–34733.

Dongwon Jung, Wenxuan Zhou, and Muhao Chen. 2025. Code execution as grounded supervision for LLM reasoning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, EMNLP 2025, Suzhou, China, November 4-9, 2025*, pages 24811–24822. Association for Computational Linguistics.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.

Jia LI, Edward Beeching, Lewis Tunstall, Ben Lipkin, Roman Soletskyi, Shengyi Costa Huang, Kashif Rasul, Longhui Yu, Albert Jiang, Ziju Shen, Zihan Qin, Bin Dong, Li Zhou, Yann Fleureau, Guillaume Lample, and Stanislas Polu. 2024. Numinamath. [https://huggingface.co/AI-MO/NuminaMath-CoT](https://github.com/project-numina/aimo-progress-prize/blob/main/report/numina_dataset.pdf).

Qintong Li, Leyang Cui, Xueliang Zhao, Lingpeng Kong, and Wei Bi. 2024. Gsm-plus: A comprehensive benchmark for evaluating the robustness of llms as mathematical problem solvers. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2961–2984.

Ziniu Li, Congliang Chen, Tian Xu, Zeyu Qin, Jiancong Xiao, Zhi-Quan Luo, and Ruoyu Sun. 2025. Preserving diversity in supervised fine-tuning of large language models. In *International Conference on Learning Representations*, volume 2025, pages 66127–66154.

Weibin Liao, Xu Chu, and Yasha Wang. 2025. Tpo: Aligning large language models with multi-branch & multi-step preference trees. In *International Conference on Learning Representations*, volume 2025, pages 26698–26720.

Weibin Liao, Xin Gao, Tianyu Jia, Rihong Qiu, Yifan Zhu, Yang Lin, Xinyu Ma, Junfeng Zhao, and Yasha Wang. 2026a. Learnat: Learning nl2sql with ast-guided task decomposition for large language models. In *International Conference on Learning Representations*, volume 2026, pages 87420–87452.

Weibin Liao, Jian-guang Lou, and Haoyi Xiong. 2026b. Hyfunc: Accelerating llm-based function calls for agentic ai through hybrid-model cascade and dynamic templating. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*, pages 819–830.

Bill Yuchen Lin, Abhilasha Ravichander, Ximing Lu, Nouha Dziri, Melanie Sclar, Khyathi Raghavi Chandu, Chandra Bhagavatula, and Yejin Choi. 2024. The unlocking spell on base llms: Rethinking alignment via in-context learning. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net.

Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, and 1 others. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*.

Junteng Liu, Yuanxiang Fan, Zhuo Jiang, Han Ding, Yongyi Hu, Chi Zhang, Yiqi Shi, Shitong Weng, Aili Chen, Shiqi Chen, Mozhi Zhang, Pengyu Zhao, and Junxian He. 2025. Synlogic: Synthesizing verifiable reasoning data at scale for learning logical reasoning and beyond. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2025, NeurIPS 2025, San Diego, CA, USA, December 2-7, 2025 / Mexico City, Mexico, November 30 - December 5, 2025*.

Haipeng Luo, Qingfeng Sun, Can Xu, Pu Zhao, Jianguang Lou, Chongyang Tao, Xiubo Geng, Qingwei Lin, Shifeng Chen, and Dongmei Zhang. 2023. Wizardmath: Empowering mathematical reasoning for large language models via reinforced evol-instruct. *arXiv preprint arXiv:2308.09583*.

Weijie Lv, Xuan Xia, and Sheng-Jun Huang. 2024. Codeact: Code adaptive compute-efficient tuning framework for code llms. *arXiv preprint arXiv:2408.02193*.

Alan Malek, Jiawei Ge, Nevena Lazic, Chi Jin, András György, and Csaba Szepesvári. 2025. Frontier llms still struggle with simple reasoning tasks. *arXiv preprint arXiv:2507.07313*.

Iman Mirzadeh, Keivan Alizadeh-Vahid, Hooman Shahrokhi, Oncel Tuzel, Samy Bengio, and Mehrdad Farajtabar. 2025. Gsm-symbolic: Understanding the limitations of mathematical reasoning in large language models. In *International Conference on Learning Representations*, volume 2025, pages 94743–94765.

Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori B Hashimoto. 2025. s1: Simple test-time scaling. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 20286–20332.

David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R Bowman. 2023. Gpqa: A graduate-level google-proof q&a benchmark. *arXiv preprint arXiv:2311.12022*.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, and 1 others. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.

Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. 2024. Hybridflow: A flexible and efficient rlhf framework. *arXiv preprint arXiv: 2409.19256*.

Aaditya K Singh and DJ Strouse. 2024. Tokenization counts: the impact of tokenization on arithmetic in frontier llms. *arXiv preprint arXiv:2402.14903*.

Xiaoxiao Sun, Mingyang Li, Kun Yuan, Min Woo Sun, Mark Endo, Shengguang Wu, Changlin Li, Yuhui Zhang, Zeyu Wang, and Serena Yeung-Levy. 2026. Do vlms perceive or recall? probing visual perception vs. memory with classic visual illusions. *arXiv preprint arXiv:2601.22150*.

Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc Le, Ed Chi, Denny Zhou, and 1 others. 2023. Challenging big-bench tasks and whether chain-of-thought can solve them. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 13003–13051.

Shubham Toshniwal, Ivan Moshkov, Sean Narenthiran, Daria Gitman, Fei Jia, and Igor Gitman. 2024. Openmathinstruct-1: A 1.8 million math instruction tuning dataset. *arXiv preprint arXiv: Arxiv-2402.10176*.

Omri Uzan and Yuval Pinter. 2026. Charbench: Evaluating the role of tokenization in character-level tasks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 33296–33304.

Peiyi Wang, Lei Li, Zhihong Shao, Runxin Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. 2024a. Math-shepherd: Verify and reinforce llms step-by-step without human annotations. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9426–9439.

Tianlong Wang, Yuhang Wang, Weibin Liao, Xin Gao, Xinyu Ma, Yang Lin, Yasha Wang, and Liantao Ma. 2026. Search for truth from reasoning: A dynamic representation editing framework for steering llm trajectories. *arXiv preprint arXiv:2606.28589*.

Xilong Wang, Hao Fu, Jindong Wang, and Neil Gong. 2025. Stringllm: Understanding the string processing capability of large language models. In *International Conference on Learning Representations*, volume 2025, pages 31535–31550.

Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, and 1 others.

2024b. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. *Advances in Neural Information Processing Systems*, 37:95266–95290.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, and 1 others. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837.

Fengli Xu, Qianyue Hao, Chenyang Shao, Zefang Zong, Yu Li, Jingwei Wang, Yunke Zhang, Jingyi Wang, Xiaochong Lan, Jiahui Gong, and 1 others. 2025. Toward large reasoning models: A survey of reinforced reasoning with large language models. *Patterns*, 6(10).

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, and 1 others. 2025a. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

Dayu Yang, Tianyang Liu, Daoan Zhang, Antoine Simoulin, Xiaoyi Liu, Yuwei Cao, Zhaopu Teng, Xin Qian, Grey Yang, Jiebo Luo, and 1 others. 2025b. Code to think, think to code: A survey on code-enhanced reasoning and reasoning-driven code intelligence in llms. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 2586–2616.

Yixin Ye, Zhen Huang, Yang Xiao, Ethan Chern, Shijie Xia, and Pengfei Liu. 2025. Limo: Less is more for reasoning. *arXiv preprint arXiv:2502.03387*.

Longhui Yu, Weisen Jiang, Han Shi, Jincheng Yu, Zhengying Liu, Yu Zhang, James Kwok, Zhenguo Li, Adrian Weller, and Weiyang Liu. 2024a. Metamath: Bootstrap your own mathematical questions for large language models. In *International Conference on Learning Representations*, volume 2024, pages 45040–45061.

Yao-Ching Yu, Chun Chih Kuo, Ye Ziqi, Chang Yucheng, and Yueh-Se Li. 2024b. Breaking the ceiling of the llm community by treating token generation as a classification for ensembling. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 1826–1839.

Jinyang Zhang, Hongxin Ding, Yue Fang, Weibin Liao, Muyang Ye, Junfeng Zhao, and Yasha Wang. 2026a. The tell-tale norm: ℓ_2 magnitude as a signal for reasoning dynamics in large language models. *arXiv preprint arXiv:2606.06188*.

Jinyang Zhang, Yue Fang, Hongxin Ding, Weibin Liao, Muyang Ye, Junfeng Zhao, Yasha Wang, and Xu Chu. 2026b. Adept: Continual pretraining via adaptive expansion and dynamic decoupled tuning. In *International Conference on Learning Representations*, volume 2026, pages 152171–152228.

Jinyang Zhang, Kexin Yang, Yu Wan, Muyang Ye, Baosong Yang, Fei Huang, Junyang Lin, and Dayiheng Liu. 2025a. NOVA-63: native omni-lingual versatile assessments of 63 disciplines. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, EMNLP 2025, Suzhou, China, November 4-9, 2025*, pages 7147–7189. Association for Computational Linguistics.

Xiang Zhang, Juntai Cao, and Chenyu You. 2024. Counting ability of large language models and impact of tokenization. *arXiv preprint arXiv:2410.19730*.

Xingyuan Zhang, Philip Becker-Ehmck, Patrick van der Smagt, and Maximilian Karl. 2023. Action inference by maximising evidence: Zero-shot imitation from observation with world models. In *Thirty-seventh Conference on Neural Information Processing Systems*.

Zhenru Zhang, Chujie Zheng, Yangzhen Wu, Beichen Zhang, Runji Lin, Bowen Yu, Dayiheng Liu, Jingren Zhou, and Junyang Lin. 2025b. The lessons of developing process reward models in mathematical reasoning. *arXiv preprint arXiv:2501.07301*.

Chujie Zheng, Zhenru Zhang, Beichen Zhang, Runji Lin, Keming Lu, Bowen Yu, Dayiheng Liu, Jingren Zhou, and Junyang Lin. 2024a. Processbench: Identifying process errors in mathematical reasoning. *arXiv preprint arXiv:2412.06559*.

Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, Zheyan Luo, Zhangchi Feng, and Yongqiang Ma. 2024b. Llamafactory: Unified efficient fine-tuning of 100+ language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, Bangkok, Thailand. Association for Computational Linguistics.

Ziyin Zhou, Xu Ji, Jianyi Zhang, Zhangchi Zhao, Xiali Hei, and Kim-Kwang Raymond Choo. 2024. Ethical considerations and policy implications for large language models: Guiding responsible development and deployment. In *International Conference on Security and Privacy in Cyber-Physical Systems and Smart Vehicles*, pages 311–329. Springer.

A Data Synthesis Pipeline

This appendix describes the complete data synthesis pipeline used to transform competitive programming problems into story-based reasoning training data. The pipeline consists of four stages, each building upon the previous one.

We used only publicly available artifacts in this work, including datasets, benchmarks, and base models released through Hugging Face or other official repositories. These resources were used in accordance with their respective licenses and terms of use, and their original sources are cited in the paper. To the best of our knowledge, all used artifacts are openly released for research use.

We decompose our data synthesis pipeline into three parts:

1. **Problem background synthesis**: The narrative context of each question is synthesized from pretraining corpora (Appendix A.2).

2. **Test case synthesis**: When test cases are not available, they are generated from the code through guided random generation(Appendix A.3).

3. **Chain-of-thought synthesis**: The CoT is synthesized from execution traces obtained via code instrumentation, where intermediate checkpoints provide process-level reasoning signals (Appendix A.4).

A.1 Stage 1: Problem Classificaton and Mathematical Kernel Extraction

Large language models (LLMs) still exhibit systematic weaknesses on tasks that require *fine-grained symbolic fidelity*, such as character-level manipulation, exact numerical computation, and step-by-step state transition under strict rules. Canonical examples include counting the number of occurrences of a specific letter in a word (e.g., how many `r`'s appear in `strawberry`) or comparing decimal numbers such as `9.9` and `9.11`. While such tasks are often trivial for programs, they remain challenging for LLMs due to tokenization artifacts, approximate pattern matching, and brittle long-chain reasoning.

To address this gap, we propose `MIMIC`, which transforms competitive programming problems into diverse, *execution-verified* reasoning training data—injecting *algorithmic reasoning primitives* that programs handle trivially but LLMs struggle with. Specifically, we collect 10,008 problems from the **open-r1/codeforces**[4], together with their accepted (AC) solutions. This source is particularly suitable for our purpose because each problem typically includes a well-specified natural language statement, detailed `input format` and `output format`, and official test cases used by the contest platform for evaluation (some included). Moreover, AC solutions have already passed the hidden and public judge cases in official Codeforces contests, providing a strong correctness guarantee. This allows us to use code as a reliable intermediate representation for synthesizing high-quality training instances, without having to independently verify the semantic validity of each solution.

We categorize each problem according to the *primary reasoning skill* required to solve it, aiming to capture those capability dimensions where code execution is naturally reliable but LLM reasoning is often error-prone. The resulting taxonomy contains five categories, shown in Table 6. Problem classification is performed by Qwen3.5-27B using type-specific prompting as in Table 6. To support type-aware data pairing in Stage 2, corpus passages from the pretraining data are independently assigned to compatible reasoning types after classification.

After classification, we further extract mathematical kernel of each problem by removing narrative background and contest-specific flavor text, while preserving the exact task semantics, constraints, and input/output requirements. The prompt used for extraction is shown below.

Prompt 1

Role: Competitive Programming Problem Refiner
Task Description. You are an efficient translator for competitive programming problems. Your goal is to remove all background stories, character names, narrative framing, and other non-essential text, and rewrite the problem as a concise and direct task description.
Instructions. I will provide a `[Problem Description]`. You should retain only the problem kernel and express it in the most concise and explicit form possible.

[4] https://huggingface.co/datasets/open-r1/codeforces

Type	Category	Subtype	Description
Type 1	**Text-Encoded Visual / Spatial Reasoning**	1.1 Grid / board / pixel encoding	Layouts represented as character matrices or ASCII art, where the solver must reconstruct a 2D structure from text and reason about adjacency, connectivity, or shape.
		1.2 Geometry / coordinates / directions	Spatial reasoning grounded in language, such as left/right/up/down relations, segment intersection, coordinate movement, or point-in-polygon logic.
		1.3 Glyph or compositional character structure	Reasoning based on visual properties of symbols or characters, including radicals, stroke counts, mirrored forms, rotated shapes, or internal composition.
		1.4 Textual charts / tabular layout understanding	Information encoded through alignment, indentation, spacing, or fixed-width formatting, such as tree structures or visually organized tables.
Type 2	**Sub-Token / Character Manipulation**	2.1 Character-level counting / extraction / replacement	Fine-grained operations on strings, including counting symbol frequency, extracting characters at specified positions, or performing local edits and substitutions.
		2.2 Character-wise mapping and ciphers	Per-character transformation rules such as Caesar shifts, ASCII-based conversion, or Base64-style encoding and decoding.
		2.3 Strict format validation	Verifying whether a string satisfies exact symbolic constraints, including delimiter placement, serial patterns, and regular-expression-like rules.
Type 3	**Precise Mathematical Computation**	3.1 Discrete and high-precision arithmetic	Exact computation over very large integers, long decimals, exponentiation, or arithmetic that cannot tolerate approximation.
		3.2 Algebraic and symbolic manipulation	Equation solving, matrix derivation, symbolic transformation, calculus-style reasoning, or other formula-driven procedures.
		3.3 Probability, statistics, and set operations	Computation involving variance, standard deviation, permutations, combinations, probability distributions, or exact set-based reasoning.
Type 4	**Large-Scale Data Processing**	4.1 Retrieval and filtering	Searching through large collections to identify records that satisfy specified conditions.
		4.2 Grouping, aggregation, and sorting	Reorganizing global data via sorting, grouping, counting, summation, or other aggregate operations.
		4.3 Structural mapping and transformation	Converting data from one representation to another, such as JSON-to-CSV conversion or flattening nested dictionaries.
Type 5	**Strict Rule / State Simulation**	5.1 Path finding and graph traversal	Reasoning over reachable states or paths, typically involving DFS, BFS, shortest-path search, or related graph procedures.
		5.2 Constraint satisfaction and backtracking	Searching for valid assignments under mutually dependent rules, such as Sudoku, N-Queens, or scheduling problems.
		5.3 Iterative and dynamical system simulation	Simulating systems whose state at step t depends exactly on step $t-1$, including cellular automata, physical processes, or code-execution-style state updates.

Table 6: Reasoning taxonomy used in Stage 1. Each problem is assigned to the category that best reflects its dominant reasoning bottleneck.

Constraints.

- **Core Only:** Do not include any preamble or explanation. Use a direct structure such as "Given . . . , find/calculate . . . "
- **Plain Talk:** Use standard competitive-programming terminology that is immediately understandable, such as *array*, *subarray*, *non-overlapping*, *maximum sum*, *path*, or *tree*.
- **Output Format:** Return the result strictly in JSON format with a single `"text"` field. Do not include any extra explanation or markdown outside the JSON.
- **Preserve Semantics:** Do not omit or alter any logic, constraints, input/output requirements, or the intended solution structure of the original problem.

Input Problem Description:
`George wants to buy a new phone but doesn't have enough money, so he works as a programmer. His boss gives him a sequence of` n `integers and asks him to choose` k `pairs of integers` $[l_i, r_i]$ `such that they represent` k `non-overlapping subarrays of length` m `to maximize the total sum . . .`

Output:
`"text": "Given a sequence of length n, choose k non-overlapping subarrays, each of length m, such that the sum of all elements in these subarrays is maximized."`

Problem Description:
`{question}`

Output Format. After reasoning, return the extracted kernel directly in JSON format:

```
{
  "text": "extracted kernel"
}
```

We further illustrate the extraction step with a concrete example. As shown below, mathematic kernal extraction removes narrative framing and contest-specific wording while preserving the exact computational objective of the original problem. This helps isolate the problem kernel and makes the resulting instance better suited for reasoning-focused synthesis.

Example 1

Mathematic Kernal Extraction To reduce competitive programming-specific narrative cues, we refine each original problem into a concise kernel that preserves only the mathematical task. For example, the original statement of *Codeforces 1304/A* (*Two Rabbits*) contains an extended story about two rabbits in a park:

"Being tired of participating in too many Codeforces rounds, Gildong decided to take some rest in a park . . . He noticed that the two rabbits were hopping towards each other . . . Will the two rabbits be at the same position at the same moment?"

After extraction, this narrative framing is removed and only the task-relevant kernel remains:

"Given integers x, y, a, b *where* $x < y$*, determine the smallest non-negative integer* t *such that* $x + at = y - bt$*. If no such* t *exists, output* -1*; otherwise, output* t*."*

This extraction step preserves the exact semantics, constraints, and intended solution structure, while removing background stories and stylistic contest phrasing that may otherwise trigger superficial dataset recall.

A.2 Stage 2: Problem Synthesis — Story Wrapping

Stage 2 rewrites each algorithmic problem into a realistic exam-style question by pairing it with an external real-world passage and using the passage as the narrative environment for the problem. The goal is to preserve the exact computational structure of the original task while removing contest-style presentation and recasting it as a self-contained question that looks like it was written for

a human test.

The external passages are sampled from **FineWebEdu**[5], a large-scale educational web corpus containing expository and informational text across diverse domains. We use FineWebEdu because its topical breadth and great education value make it a good source of realistic background material for written exam questions.

A key challenge in this stage is that not every passage can serve as a natural wrapper for every problem. If the passage is incompatible with the problem's reasoning type, the resulting rewrite often becomes forced: the story is only loosely attached to the underlying logic, important quantities or structures must be invented, or the final question retains obvious traces of programming formulations. We therefore perform *type-aware pairing*: before synthesis, we classify corpus passages by the kinds of reasoning structures they can support, and only pair a problem with passages compatible with its assigned type from Stage 1.

This filtering is especially important for the categories we target. Type 2 problems require coherent natural-language prose that can plausibly serve as the exact string being counted, transformed, or validated. Type 3 problems benefit from passages that already contain concrete extractable values, so that the synthesized question can reuse naturally occurring numbers or its occurring place rather than fabricated ones. Type 4 problems require structured entries or records that support filtering, grouping, sorting, or aggregation. Type 5 problems require passages with explicit sequential rules or state transitions, for example, procedural instructions, staged workflows, biological cycles, or turn-based processes, so that the narrative already contains a natural notion of stepwise evolution. For this reason, we classify FineWebEdu passages with a strict prompt and use the resulting labels to guide problem–passage matching.

The full prompt used for corpus classification is shown below:

Prompt 2

Strict Corpus Classification Prompt

You are a strict corpus classifier for exam problem synthesis. A text passage will be provided along with rule-based type hints from a fast heuristic scan. Your task is to determine which problem types (1–5) this text is **strictly** suitable for as background context or direct input.

Type Definitions

- **Type 1 – Visual/Spatial Reasoning**: The text explicitly describes a grid, map, floor plan, coordinate system, or directional/positional relationships between objects (e.g., "the room to the left of the hall", "cell (2,3) contains a wall"). The spatial structure must be clear enough to reconstruct a 2D layout.
- **Type 2 – Character-Level Manipulation**: The text is coherent, continuous natural-language prose ($\geq$ 4 complete sentences) with no dominant numeric or structural content. It is suitable as raw input for character-level operations: counting specific letters, detecting substrings, verifying text patterns, or string formatting tasks.
- **Type 3 – Precise Mathematical Computation**: The text contains **at least 2 concrete numeric values** (measurements, statistics, prices, counts, dates, percentages) that are human-computable and can be directly extracted and used as parameters in a calculation problem.
- **Type 4 – Data Processing**: The text contains a **structured dataset**: a table with rows and columns, or a list with **at least 3 entries** each having multiple named attributes (e.g., name + value + category). The data is suitable for filtering, sorting, grouping, or aggregation operations.
- **Type 5 – State/Rule Simulation**: The text describes a **step-by-step process, iterative rules, or explicit state transitions** with enough detail to be modeled as a simulation (e.g., game rules with clear turn mechanics, a biological lifecycle with defined stages, a procedural algorithm described in prose).

Strictness Rules

- Mark a type as suitable **ONLY** if the text **clearly and unambiguously** provides what that type requires.

[5] https://huggingface.co/datasets/HuggingFaceFW/fineweb-edu

- A text may be suitable for **zero, one, or several** types.
- Rule-based hints (fast heuristic): `[rule_hints]` — treat as a starting point; override if the evidence in the text does not support a hint, or add a type the heuristic missed.
- **Do NOT** mark Type 1 unless the text contains actual spatial/positional structure, not just vague location words.
- **Do NOT** mark Type 2 if the text is primarily numeric, tabular, or highly fragmented.
- **Do NOT** mark Type 3 unless the numeric values are specific and extractable (e.g., "37.5°C", "142 patients", "$2,380"), not vague estimates.
- **Do NOT** mark Type 4 unless the structured entries are clearly delineated with multiple attributes per entry.
- **Do NOT** mark Type 5 unless the sequential rules are explicit and complete enough to simulate step-by-step.

Output Format

Respond with **only** a valid JSON object, no other text:

```
{
  "suitable_types": [integers],
  "reasons": {
    "<type_number>": "<>"
  }
}
```

Text to Classify

`{text}`

Once a compatible passage is selected, we prompt the model to rewrite the original problem into a realistic exam question. The synthesis prompt is built around three core requirements. First, the *corpus is the world*: the domain, entities, terminology, and contextual details should come from the paired FineWebEdu passage rather than from invented filler text. Second, the rewrite must contain *zero programming traces*: variables, input/output phrasing, and algorithmic language are removed and replaced with natural prose. Third, the rewrite must preserve *logical fidelity*: the mathematical structure, decision rules, and stopping conditions of the original problem remain unchanged, even though they are expressed through a realistic scenario.

Two additional constraints are particularly important for data quality. We require that *all necessary information be embedded directly in the prose narrative*, so that the question is fully understandable without any contest-style input block. We also encourage *rich contextual wrapping*: the passage should contribute substantial background detail, including irrelevant but realistic information, so that the result resembles a genuine exam reading task rather than a thinly disguised programming prompt. The full synthesis prompt is given below.

Prompt 3

Type-Aware Story-Wrapping Prompt

You are a top-tier exam question designer. Your task is to rewrite an algorithmic problem as a realistic written exam question that looks like it came from a human aptitude test, economics exam, or science assessment — never from a programming contest. You will receive:

- A **corpus passage** (real-world text providing narrative context and potentially real data)
- An **original algorithmic problem** (defines the exact computation logic and I/O format)
- A **problem type** (tells you how to fuse the corpus with the logic)

Universal Rules (apply to every type)

1. **The corpus is your world**: The scenario, characters, domain, and vocabulary of the exam question must come from the corpus. Never invent a fictional world when the corpus provides a real one.
2. **Zero programming traces**: Ban all of — variable names (`n`, `x`, `i`), "input/output format", "algorithm", "time complexity", "given a positive integer", "print the answer". Replace with natural prose.

3. **Concrete nouns replace variables**: Instead of "integer `n`", write "the number of monitoring stations", "the observation period in months", "the total number of affected residents" — nouns extracted or inspired by the corpus.

4. **Logical fidelity**: The underlying mathematical model, decision rules, and stopping conditions must remain identical to the original problem. You may rephrase, never re-derive.

5. **Self-contained question**: A test-taker with no programming knowledge should be able to read the question and understand exactly what is being asked.

6. **ALL data embedded in prose (CRITICAL)**: Every numerical value, list, table, or parameter that the solver needs MUST appear directly within the problem narrative as natural text.

7. **Rich context with distraction (CRITICAL)**: Incorporate a portion of the corpus passage into the problem, with some contextual detail not directly needed for solving.

8. **Never reveal the mathematical model**: Do NOT write equivalences like "this is equivalent to computing X" or "in other words, calculate Y^n".

Type-Specific Fusion Strategy
Type `{type_id}` requires the following approach:
`{type_instruction}`
Output Format
Respond with **only** a valid JSON object — no markdown, no explanation:

```
{
  "problem": "<A realistic question>",
   "input": "<stdin for AC code>"
}
```

IMPORTANT: The "input" field must contain values that are EXACTLY derivable from the "problem" text. A human reading only the "problem" field should be able to extract all the data needed and arrive at the same input values. The "input" field is only used internally for automated answer generation — it is never shown to the test-taker.
Inputs
Corpus passage: `{text}`
Original algorithmic problem (logic + I/O format): `{problem}`

Finally, for each source problem, we generate 20 story variants by pairing it with different compatible FineWebEdu passages, yielding roughly 200K synthesized questions. These variants share the same executable logic but differ in domain, discourse, and surface realization, which increases the diversity and generalizability of the resulting training data.

After the initial story-wrapping step, we perform a lightweight *narrative refinement* pass to improve coherence. In early synthesis outputs, we frequently observe a "two-block" artifact: the model first reproduces a large chunk of corpus text, and then abruptly switches into the actual question through a transition such as "consider the following case" or "in a separate task." Although the underlying logic is correct, this structure makes the result feel mechanically assembled rather than naturally written.

To address this, we apply a second-pass editing prompt that treats the initial synthesis as a draft and rewrites it into a more unified exam question. The goal is not to alter the computation, but to improve narrative flow: the background and the task should belong to the same story, each sentence should either motivate the question or provide relevant information, and the final wording should read like a coherent human-written exam item rather than a corpus passage followed by an attached exercise. Importantly, this refinement step is constrained to preserve the hidden `input` exactly, so the executable semantics and final answer remain unchanged.

Compared with the initial drafts, this refinement step substantially reduces redundant context and improves story–task integration. In our data, the average problem length decreases by approximately 35% while narrative coherence improves noticeably. The full prompt is given below.

Prompt 4

Narrative Refinement Prompt

You are an expert exam question editor. You will receive a draft exam question that was machine-generated by fusing a real-world corpus passage with an algorithmic problem. Your job is to REFINE the draft into a polished, natural question where the background story and the computational task are seamlessly woven together.
You will receive:

- The **draft problem** (the machine-generated question to improve)
- The **original input** (the stdin that produces the ground-truth answer — you must preserve this exactly)
- The **corpus passage** (the original real-world text used for context)

What's Wrong with Typical Drafts
Most drafts suffer from a "two-block" structure:

1. A long block of corpus text (copied almost verbatim)
2. A sudden transition sentence ("In a separate task...", "Consider a specific...", "To better visualize...") followed by the actual question

This feels like two unrelated articles glued together. The reader can clearly see where the background ends and the real question begins.
Your Refinement Goals

- **Goal 1: Narrative Unity — The story IS the problem.** The background and the question must be part of the same story. The reader should not feel a seam between context and task.
- **Goal 2: Causal Chain — Every sentence earns its place.** Each piece of background should either explain why the problem exists or provide data needed to solve it. Remove irrelevant copied context.
- **Goal 3: Characters and Motivation — Someone needs this answer.** You can introduce a concrete agent from the corpus world (e.g., researcher, inspector, technician, student, manager) who has a reason to need the result.
- **Goal 4: Appropriate Length.** The refined question should be 150–400 words. Do not pad with irrelevant detail.
- **Goal 5: Creative Integration over Literal Copying.** Rephrase, reorganize, and selectively use the corpus rather than copying large blocks verbatim.

Hard Constraints (DO NOT VIOLATE)

1. **The answer must not change.** The `input` must remain EXACTLY the same as the original.
2. **All solving data must appear in the text.** Every value needed to derive the `input` must be explicitly stated in the problem narrative.
3. **No programming traces.** No variable names, no "input/output format", no "algorithm", no "given a positive integer".
4. **No revealing the math model.** Do not explain the hidden formula or say what the problem is "equivalent to".

Output Format
Respond with **only** a valid JSON object — no markdown, no explanation:

```
{
  "problem": "<refined question>",
}
```

Inputs
Draft problem: `{draft_problem}`
Corpus passage (for reference — use selectively): `{corpus}`

To illustrate the effect of Stage 2 synthesis and the subsequent refinement step, we present two representative examples below. We choose them because they capture two distinct but common failure modes in raw generation. The first is a Type 2 character-level problem, where naive synthesis often leaves a visible seam between a borrowed passage and the actual decoding task. The second is a Type 4 data-processing problem, where long descriptive context can easily overwhelm the operational question. In both cases, refinement preserves the original executable input while making the final question more coherent, natural, and exam-like.

Example 2

Case 1: Type 2 — Character-Level Manipulation (Borze decoding)
Original problem. Decode a Borze string, where `.`, `-.`, and `--` map to digits 0, 1, and 2.
Corpus context. Historical and industrial background of Épinal, France.

Before refinement. The draft first presents a long encyclopedia-style description of Épinal, including its industries, wartime history, and cultural significance, and only then abruptly introduces a separate paragraph about archivists decoding an old telegraph log. Although the decoding task is correct, the story and the question feel like two unrelated texts pasted together.

After refinement. The rewritten version turns the entire problem into a single archival scenario: a regional library in Épinal is digitizing early 20th-century telegraph records related to textile production, and the recovered string `-.-.-.-.` must be decoded using the local three-symbol code. This removes the visible transition between "background" and "task" and makes the character-level operation feel naturally motivated by the narrative.

Case 2: Type 4 — Data Processing (frequency-based assignment)
Original problem. Assign distinct prices to item types according to frequency so as to minimize and maximize the total cost.
Corpus context. Butterfly species records in Carson County, Texas.

Before refinement. The draft contains a long ecological description of multiple butterfly species and habitats, followed by a sudden switch to a budgeting problem involving six grant amounts and a short observation log. The resulting question is solvable, but most of the biological background does not directly support the data-processing task, producing a clear narrative seam.

After refinement. The refined version recasts the task as a conservation funding decision: county officials allocate distinct grant amounts to butterfly species based on how often each species appears in a field survey. The survey log and the six funding amounts are embedded directly in the narrative, and the computational objective—finding the minimum and maximum possible total allocation under different assignments—now arises naturally from the administrative scenario.

In both examples, the refinement step preserves the hidden `input` exactly while substantially improving narrative unity and reducing the "two-block" artifact.

A.3 Stage 3: Test Case Synthesis and Narrative Input Fusion

The story-wrapped questions generated in Stage 2 provide only one executable instance per problem, namely the input values embedded in the rewritten narrative. Test data obtained from real-world corpora is often overly simplistic; on the one hand, there is a lack of data within the corpus, and on the other, the model itself tends to favour simpler numbers. (Details in Appendix D) This is insufficient for broad training coverage. In addition, while Codeforces problems are validated by official judge cases, those test cases are not always fully available in the released data, particularly the more challenging test cases. More importantly, many inputs that are useful for our setting (such as adversarial symbolic strings, compact edge cases, or carefully structured numeric patterns) do not naturally appear in ordinary corpus passages. We therefore synthesize additional test inputs and then fuse them back into the existing story templates.

Guided random test synthesis. Our test generation process is guided by the accepted code rather than by uninformed random sampling. Specifically, we ask the LLM to read the AC solution and identify its key *decision points*: branch conditions, comparison patterns, loop boundaries, special-case handling, and value configurations that may trigger qualitatively different execution paths. This analysis is performed by the LLM itself during generation. For each decision point, the model specifies both the triggering condition and a corresponding *guided mode*, a structured pattern of inputs likely to hit that behavior reliably.

This setup is intended to uncover *small but meaningful* logical vulnerabilities rather than to maximize raw computational burden. For example, the

model may identify patterns such as singleton inputs, equal neighboring values, alternating structures, periodic layouts, or a single extreme outlier. These patterns are then turned into explicit test-generation strategies.

Three levels of test inputs. Using the decision-point analysis, we synthesize three levels of test inputs for each problem.

Level A: Minimal sanity cases. These are small, mostly hard-coded cases designed to verify basic correctness. They typically use tiny sizes and simple values, and cover obvious edges such as singleton inputs, minimal valid structures, or degenerate small cases. We generate 5 such cases per problem.

Level B: Hard-coded adversarial cases. These cases are constructed to hit the most important decision points directly. From the decision-point analysis, the model selects five critical branches and creates one deterministic case for each. These are intentionally non-random and are meant to trigger specific boundary behaviors, special cases, or branch-sensitive patterns. We generate 5 such cases per problem.

Level C: Guided random cases. These cases provide broader coverage while remaining structured. The decision points are converted into a set of generation modes, such as `all_equal`, `alternating`, `nearly_sorted`, `large_spike`, or `periodic`. Each mode has its own generator logic, and sampling rotates across modes to ensure diverse behavioral coverage. To keep execution efficient and avoid turning the task into unnecessarily heavy arithmetic, we cap input sizes at moderate ranges (e.g., $n \leq 50$, values $\leq 10^5$). We generate 10 guided-random cases per problem.

We omit the full prompt used for test synthesis. Unlike the other prompts in our pipeline, this prompt contains substantial program-specific content, including code formatting requirements, judge-dependent output templates, and execution-safety constraints, which make it considerably longer. As the central design principles have already been introduced in the preceding sections, reproducing the entire prompt here would add implementation detail without further clarifying the method. To make the test-synthesis procedure more concrete, we provide one representative example below. This example illustrates how the model analyzes an accepted solution, identifies branch-sensitive decision points, and turns them into a small set of adversarial and guided-random test modes. Importantly, the goal is not to maximize raw input size, but to generate *small, sharp* cases that are likely to expose logical mistakes in symbolic reasoning or state updates.

Example 3

Example: Decision-point-driven test synthesis for Codeforces 1426/F

Problem summary. The input is a string over a, b, c, and ?. Each ? can be replaced by any of a/b/c. The task is to compute the total number of abc subsequences across all resulting strings, modulo $10^9 + 7$.

Core AC logic. The accepted code maintains four DP states:

- `seq`: number of partial strings generated so far ($= 3^{\#?}$),
- `a`: total number of a subsequences,
- `ab`: total number of ab subsequences,
- `out`: total number of abc subsequences.

For fixed characters, the update is straightforward:

- `'a'`: `a += seq`
- `'b'`: `ab += a`
- `'c'`: `out += ab`

The key difficulty lies in the `'?'` branch, where all three replacements must be merged carefully. In particular, the code must preserve the old value of a before scaling, and must use the old ab when updating `out`.

Identified decision points. By reading the AC code, the model extracted five branch-sensitive behaviors:

1. the `'a'` branch, where `a += seq` must use the current number of partial strings;
2. the `'b'` branch, where `ab += a` depends on the correct current value of a;
3. the `'c'` branch, where `out += ab`;
4. the `'?'` branch, where update order is crucial, especially the preservation of the old a and the use of the pre-scaled ab;
5. the initialization `seq = 1`, without which the entire DP collapses.

Level A: minimal sanity-check cases. The model first generated a small set of short inputs whose outputs are easy to verify by hand while still exercising distinct DP behaviors:

- `abc`: the simplest nonzero case, yielding exactly one `abc` subsequence,
- `acb`: a short reverse-order case, where the answer should be zero,
- `a?c`: checks whether a single wildcard can correctly complete the middle `b`,
- `?bc`: checks whether a wildcard at the beginning can contribute the leading `a`,
- `ab?`: checks whether a wildcard at the end can contribute the trailing `c`.

Level B: hard-coded adversarial cases. Based on these decision points, the model constructed five deterministic test cases:

- `???`: pure wildcards; exposes incorrect update ordering in the `'?'` branch,
- `?a?`: checks whether `seq` is propagated correctly through wildcard expansion and then reused by a fixed `'a'`,
- `a??`: verifies whether an early `a` is correctly expanded across multiple wildcard rounds,
- `?b?`: stresses the formation and accumulation of `ab`,
- `?????`: tests multi-round wildcard updates and modular accumulation.

Level C: guided-random modes. The model then converted the decision points into five reusable sampling modes (The specific data is generated randomly under these patterns):

- `wildcard_heavy`: about 70% of positions are `'?'`, to stress repeated complex updates;
- `fixed_sequence`: only a/b/c, to validate the base DP logic;
- `interleaved`: strict alternation between `'?'` and fixed characters, to force frequent switching between simple and complex branches;
- `prefix_suffix`: wildcards concentrated near the beginning and end, to test boundary propagation;
- `reverse_pattern`: strings dominated by $\mathtt{c} \to \mathtt{b} \to \mathtt{a}$ order, where the true count should remain very small and spurious overcounting is easier to detect.

The generated lengths were intentionally restricted to roughly 3–50, following our "small but sharp" principle: the synthesized tests are designed to stress logical correctness rather than brute-force numerical scale.

Execution and gold outputs. Every synthesized input is executed against the original accepted code in a sandboxed environment to obtain the deterministic gold output. This guarantees that the supervision signal comes from verified program execution rather than from an LLM's own answer generation. We also retain available official examples or released test cases when present.

Narrative input fusion. After generating new inputs, we convert them into additional story-based questions by reusing the narrative templates from Stage 2. Given an existing story-wrapped problem and a newly synthesized input, we ask the model to rewrite the question so that the new values are naturally woven into the same scenario. This step is necessary because many synthesized inputs. For example, unusual symbol strings, boundary coordinates, or structured numeric sequences do not occur in the original corpus passage and must therefore be inserted explicitly into the prose.

The fusion prompt requires the model to identify which parts of the narrative correspond to the original executable input, replace them with the new values, and update all dependent textual details so that the story remains internally consistent. For example, if a new input changes the number of entities, then all related counts, names, and enumerations in the narrative must be updated as well. At the same time, the story world, characters, motivation, and question logic must remain unchanged. The result is a new natural-language question from which the updated executable input can again be fully recovered.

The full prompt used for this step is shown below.

Prompt 5

Narrative Input Fusion Prompt

You are an expert exam question editor. You will receive a story-based exam question (which already embeds some specific data values) and a NEW set of input data that must replace the original values. Your task is to rewrite the question so that the NEW data is naturally woven into the existing narrative, while preserving the story, characters, and scenario exactly as they are.

What You Must Do

1. **Identify the data points** in the original question that correspond to the original input (dimensions, counts, coordinates, strings, sequences, etc.)
2. **Replace them** with the corresponding values from the new input, adjusting the narrative text naturally (e.g., "a 6-meter by 6-meter room" becomes "a 10-meter by 8-meter room" if the new input is `10 8 4`)
3. **Maintain narrative consistency**: when data changes affect quantities, update ALL related narrative elements accordingly. For example, if the number of items changes from 6 to 8, you must also update the count mentioned in prose, add or remove named entities to match, and ensure all enumerated items in the text match the new count.
4. **Keep the core story unchanged**: the setting, characters' roles, scenario, motivations, and the question being asked must remain identical
5. **Ensure all new data values appear explicitly in the text** — a reader should be able to extract the new input from the rewritten question alone

Hard Constraints

1. **The story and question logic must not change.** Only the embedded data values change.
2. **Every value from the new input must appear in the output text.** Do not omit any data point.
3. **No programming traces.** No variable names, no "Input:", no code formatting.
4. **Preserve the narrative style and length** of the original question. Do not make it significantly shorter or longer.
5. **If the new input contains strings or text**, embed them naturally as quotes, excerpts, or referenced content within the narrative.

Output Format

Respond with **only** a valid JSON object:

```
{
  "problem": "<question with new data>"
}
```

Inputs

Original story-based question: `{problem}`
Original input data (embedded in the question above): `{original_input}`
NEW input data (must replace the original): {new_input}

By combining decision-point-driven test synthesis with narrative input, we expand each source problem from some simple story-based examples into a series of executable, more challenging and comprehensive test sets, thereby probing a wider range of logical behaviours.

A.4 Stage 4: Code Instrumentation, Trace Collection, and CoT Synthesis

The final stage of our pipeline constructs the *output side* of the training data, i.e., high-quality reasoning trajectories aligned with the story-based questions produced earlier. A key observation is that accepted code naturally contains a form of process supervision: during execution, it passes through meaningful intermediate states, branch decisions, and variable updates that reflect how the problem is solved. Instead of relying on expensive human annotation, search-based reasoning distillation, or a much stronger teacher model to produce step-by-step solutions, we expose this latent process signal by instrumenting the code with sparse diagnostic breakpoints and then convert the resulting execution traces into natural-language chains of thought.

Sparse instrumentation over AC code. We begin from the original accepted solution and ask an LLM to insert purely observational diagnostic outputs. The instrumentation is strictly read-only: it

may only add printing statements and must never modify, delete, or reorder any original code. All diagnostic outputs are prefixed with `[TRACE]` so that they can be separated from the program's official answer after execution.

The main design challenge is to expose enough intermediate structure to make the computation interpretable, while keeping trace volume manageable. We therefore encourage the model to insert trace points only at semantically meaningful locations:

- **After preprocessing**: once sorting, grouping, graph construction, or other data preparation is complete;
- **When key variables are updated**: for example, when the current best value changes or when an important counter is increased;
- **At major branch decisions**: to record which path was taken and why;
- **At phase boundaries**: for multi-stage algorithms such as preprocess → optimize → aggregate;
- **Before final output**: to summarize the decisive quantities behind the answer.

Conversely, we avoid tracing every innermost-loop iteration, temporary helper variables, or computations that do not materially affect the final result. This makes the trace naturally sparse and more useful as a reasoning scaffold.

The full prompt used for code instrumentation is shown below.

Prompt 6

Code Instrumentation Prompt

You are a code analysis and debugging expert. Your task is to insert diagnostic output statements into a program that has passed all test cases (AC code), so that the program prints key intermediate computation results during execution. These intermediate results will help humans understand the program's complete problem-solving process.

You will receive:

- An **algorithm problem** (describing the problem background, input/output format, and examples)
- An **AC code** (a correct program that has passed all test cases)

Core Principles

1. **Read-only instrumentation**: You may only ADD output statements — never modify, delete, or reorder any existing code. Every inserted statement must be purely observational.
2. **[TRACE] prefix**: All inserted diagnostic output must start with `[TRACE]`.
3. **Structured format**: Each TRACE line must follow the format `[TRACE] event_label | key1=value1 | key2=value2`.
4. **Output volume control**: No more than 20 TRACE lines total; never print unconditionally inside the innermost loop; prefer event-driven output.
5. **Match the original code's output method**: use the same printing style as the original program whenever applicable.

Instrumentation Location Guidance

The following are typically valuable observation points:

- After input is fully read
- After data preprocessing
- When a key variable is updated
- At important branch decisions
- At algorithm phase transitions
- Before final output

You do not need to cover every category, select the 5–15 observation points most critical to understanding the specific algorithm.

Output Format

Output only a valid JSON object:

```
{
  "analysis": "<summarization>",
  "trace_plan": [
    {
    "location": "<insertion point>",
      "event": "<event label>",
    "variables": "<list of values>",
      "reason": ""
    }
  ],
```

```
  "instrumented_code": ""
}
```

Important:

- instrumented_code must be complete runnable code
- After filtering out all `[TRACE]` lines, the program's output must be identical to the original output
- Each entry in `trace_plan` must correspond one-to-one with an actual TRACE statement in `instrumented_code`

Input
Algorithm problem: `{problem}`
AC code:

```
{language}
{code}
```

Trace collection. We execute the instrumented programs on the verified test inputs synthesized in the previous stage. The raw stdout therefore contains two parts: the ordinary final answer and the inserted `[TRACE]` lines. Since instrumentation is read-only, filtering out all trace lines should recover exactly the original program output. This gives us execution-aligned intermediate states for the same inputs that were already used to construct story-based questions.

Trace-guided CoT synthesis. Given a story-based exam question, the gold answer, the instrumented code, and the raw execution output, we then synthesize a natural-language chain of thought. The process has two phases. First, we verify that the execution output is consistent with the known correct answer after removing the `[TRACE]` lines. If the answer does not match, we discard the sample rather than attempting to produce reasoning over an inconsistent trace. Second, if verification succeeds, we ask the model to generate a numbered reasoning chain in natural language.

The reasoning prompt explicitly requires the model to use the execution traces only as latent guidance, while expressing the final explanation entirely in the story's language. Each step should perform exactly one logical operation, such as identifying relevant quantities, making a local observation, carrying out a single calculation, or drawing an intermediate conclusion. The last step must state the final answer clearly. This format makes the resulting output suitable not only for SFT, but also for later RL use cases such as step-level reward assignment or process supervision.

The full prompt used for CoT synthesis is shown below.

Prompt 7

Trace-Guided CoT Synthesis Prompt

You are an expert reasoning assistant. You will receive a story-based exam question, the correct answer, the solution code (with diagnostic instrumentation), and the raw execution output from running that code on the given input.
Your job has two phases:

Phase 1: Verify Output Correctness
The raw execution output contains both `[TRACE]` diagnostic lines and the program's actual answer. Identify the actual answer by filtering out all `[TRACE]` lines.
Compare this actual answer against the correct answer:

- For integers/strings: exact match after trimming whitespace
- For floating-point numbers: relative error $< 1\%$

If they DO NOT match, respond with:

```
{
  "verified": false,
}
```

Do not generate any reasoning in this case.

Phase 2: Generate Step-by-Step Reasoning (only if verified)
Write a reasoning chain list that explains how to solve the problem. Use the code logic and execution traces as your guide, but express everything in the story's language.
Step Format

- Each step performs exactly ONE logical operation
- Show the actual computation — formulas, substitutions, arithmetic
- Every step must advance toward the final answer

Using the Code and Traces

- Read the code to understand the algorithm's strategy
- Use `[TRACE]` lines to identify key intermediate results

- Select the most informative checkpoints and fill in the reasoning between them
- Translate all technical concepts into the story's terminology

Final Step The last step must clearly state the final answer.

Output Format

```
{
  "verified": true,
  "num_steps": <#reasoning steps>,
  "reasoning": ["", "", ""]
}
```

What NOT to Do

- No programming terminology
- No references to the code or traces
- No mentions of Codeforces or competitive programming
- No hedging

Inputs

Exam question: `{problem}`
Correct answer: `{answer}`
Instrumented solution code:

`{instrumented_code}`

Raw execution output (includes `[TRACE]` lines and the actual answer): `{raw_stdout}`

Resulting supervision signal. This stage yields reasoning traces that are grounded simultaneously in three sources: the executable correctness of the AC solution, the intermediate structure exposed by instrumentation, and the natural-language scenario established by the story-wrapped problem. The resulting data are particularly useful for downstream supervised fine-tuning and reinforcement learning, because they provide explicit multi-step decompositions that can be treated as full targets, partial trajectories, or step-level supervision units. To illustrate the final output of our trace-guided reasoning pipeline, we show a representative positive example below. This case is particularly suitable because the execution trace exposes a compact but informative intermediate state, the sorted order of the text fragments, and the synthesized reasoning successfully translates that procedural signal into a natural-language explanation grounded in the story. The result is a coherent multi-step solution that remains faithful to both the executable logic and the narrative setting.

Example 4

Example: Trace-guided CoT synthesis for a story-based string ordering problem
Story-based question. The Interslavic Committee is cataloging manuscript fragments into a single archive identifier. Five recovered fragment codes — `zb`, `f`, `z`, `qk`, and `q` — must be arranged so that their concatenation forms the lexicographically smallest possible string.

Execution trace highlights.

```
[TRACE] n=5 | S=[zb,f,z,qk,q]
[TRACE] S=[f,qk,q,zb,z]
[TRACE] result_string=fqkqzbz
[TRACE] answer=fqkqzbz
```

Synthesized CoT.

- **Step 1:** The five fragment codes are `zb`, `f`, `z`, `qk`, and `q`, and they must be ordered into one final archive identifier.
- **Step 2:** The correct strategy is to decide the order of two fragments by comparing the two possible concatenations. For fragments `x` and `y`, if `xy` is lexicographically smaller than `yx`, then `x` should appear first.
- **Step 3:** Applying this rule to `f` and `qk`, we compare `fqk` with `qkf`. Because `fqk` is smaller, `f` comes before `qk`.
- **Step 4:** Next, compare `qk` with `q`. Here `qkq` is smaller than `qqk`, so `qk` must come before `q`.
- **Step 5:** Then compare `q` with `zb`. Since `qzb` is smaller than `zbq`, `q` should come before `zb`.
- **Step 6:** Finally, compare `zb` with `z`. Because `zbz` is smaller than `zzb`, `zb` should come before `z`. These pairwise decisions determine the full order: `f`, `qk`, `q`, `zb`, `z`.
- **Step 7:** Joining the fragments in this order gives the smallest possible archive identifier: `fqkqzbz`.

This example shows how sparse execution checkpoints can be converted into a human-readable reasoning chain without exposing

programming terminology. The trace provides the hidden procedural scaffold, while the final CoT remains fully grounded in the archive-management narrative.

B Evaluation Setting

B.1 Evaluation Framework

We conduct all evaluations using the `lm-evaluation-harness` framework (Gao et al., 2024), which provides standardized task implementations, prompt templates, answer extraction, and metric aggregation. All experiments are executed via the `vLLM` (Kwon et al., 2023) backend to ensure efficient batched generation and consistent decoding behavior across models. For code benchmarks (HumanEval, MBPP, and CRUXEval), we use a separate execution-based evaluation pipeline with sandboxed code execution and unit-test verification according to their official repository.

B.2 Benchmarks

To comprehensively evaluate both the reasoning abilities targeted by our method and potential side effects on other capabilities, we select benchmarks spanning three categories: *mathematical reasoning*, *general reasoning and knowledge* and *code generation*. Table 7 provides a summary of all benchmarks.

B.3 Metric Definition and Subset Aggregation

Per-task accuracy. For each task (or subtask) with N evaluation instances, we compute accuracy as:

$$\mathrm{Acc} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{I}\left[\hat{y}_i = y_i\right], \tag{1}$$

where y_i is the ground-truth answer, $\hat{y}_i$ is the extracted model answer, and $\mathbb{I}[\cdot]$ is the indicator function.

`weight_by_size` aggregation for benchmarks with subsets. Some benchmarks (e.g., BBH, MMLU-Pro) consist of multiple subsets $\{s\}_{s=1}^{S}$ with different sizes N_s. Following lm-evaluation-harness's `weight_by_size` strategy, we aggregate subset accuracies by weighting each subset by its number of examples:

$$\mathrm{Acc}_{\text{weighted}} = \sum_{s=1}^{S} w_s \cdot \mathrm{Acc}_s, \quad w_s = \frac{N_s}{\sum_{j=1}^{S} N_j}, \tag{2}$$

where Acc_s is computed using Eq. (1) on subset s.

B.4 Inference Configuration

We use `temperature=0.6`, `top_p=0.95`, `top_k=20`, `repetition_penalty=1.05`, `min_p=0`, and `max_gen_toks=4096`. These values follow the officially recommended decoding configuration for the `Qwen3` series *think* models. For `Qwen3` series *non-think* models, we use `temperature=0.7`, `top_p=0.8`, `top_k=20`, `repetition_penalty=1.5`, and `max_gen_toks=4096`. Since the `DeepSeek-R1-Distill-LLaMA` family does not provide an officially recommended inference setting for reasoning-focused decoding, we apply the same decoding configuration to ensure consistent evaluation across model families. As the tests involve randomness, all the results we present are the mean of six independent tests (mean@6).

C Training Setting

This section provides comprehensive details of our training setup, including model specifications, hyperparameters, and dataset configurations for both the Supervised Fine-Tuning (SFT) and Reinforcement Learning (RL) stages.

C.1 Base Models

We conduct experiments on three instruction-tuned models of comparable scale, spanning two distinct reasoning paradigms: hybrid thinking/non-thinking models and reasoning-distilled models.

Model Selection. To evaluate the generalizability of our method across different model families and its scalability within a specific architecture, we select two representative dense models: **Qwen3** (Yang et al., 2025a) nonthink models and **DeepSeek-R1-Distill-Llama-8B** (Guo et al., 2025). Specifically, we employ two variants from the Qwen3 family (4B and 8B) to investigate the impact of model scaling. Meanwhile, DeepSeek-R1-Distill-Llama-8B, built on the Llama-3.1 architecture with distilled reasoning capabilities, serves as a distinct baseline to assess cross-architecture robustness. For consistent evaluation, we disable Qwen3's native "thinking mode" to focus on direct response generation, while retaining DeepSeek's inherent chain-of-thought behavior to compare different reasoning paradigms.

Benchmark	HF Dataset	Dataset Summary	Primary Abilities Tested
Mathematical Reasoning			
GSM8K	gsm8k	Grade-school math word problems (1,319 test) requiring 2–8 steps of elementary arithmetic (Cobbe et al., 2021).	Multi-step arithmetic reasoning; text-to-equation mapping.
GSM-Plus	GSM-Plus	Adversarially augmented GSM8K (Li et al., 2024) with numerical perturbation, structural rephrasing, and distractors to reduce shortcut biases.	Robust numerical reasoning under perturbation.
MATH	competition_math	5,000 competition-level problems across 7 subjects (algebra, geometry, number theory, etc.) drawn from AMC/AIME contests (Hendrycks et al., 2021).	Formal math reasoning; symbolic manipulation; multi-step proofs.
AIME	AIME_2024 AIME_2025	60 problems from AIME 2024 and AIME2025 (I+II) (Zhang et al., 2023); integer answers in 0–999 requiring creative multi-step reasoning in algebra, combinatorics, and geometry.	Competition-level mathematical problem solving.
General Reasoning & Knowledge			
BBH	bbh	23 challenging BIG-Bench tasks where LMs previously fell below human performance (Suzgun et al., 2023), spanning logical, symbolic, linguistic, and algorithmic problem types.	Logical reasoning; compositional generalization; symbolic manipulation.
ARC-C	ai2_arc	Challenge split (1,172 questions) of the AI2 science exam benchmark (Clark et al., 2018), filtering for questions that defeat retrieval-based methods.	Scientific reasoning; commonsense knowledge; multi-hop inference.
DROP	drop	Reading comprehension requiring discrete operations (addition, counting, sorting) over paragraphs (Dua et al., 2019).	Numerical reasoning over text; discrete arithmetic over passages.
GPQA	gpqa	Graduate-level "Google-proof" science questions (physics, chemistry, biology) written by domain experts (Rein et al., 2023).	Expert-level scientific reasoning; deep domain knowledge.
MMLU-Pro	MMLU-Pro	Enhanced MMLU with 10 options and stronger contamination resistance across academic and professional subjects (Wang et al., 2024b).	Broad knowledge; domain generalization; fine-grained discrimination.
MIMIC	Ours	4,695 held-out problems from our pipeline, stratified by difficulty rating, problem type (5 categories), and source level (A/B/C tiers).	Character-level operations; precise computation; rule-based simulation.
Code Generation			
HumanEval	humaneval	164 hand-crafted Python problems with docstring specifications (Chen et al., 2021); correctness verified by executing against hidden unit tests.	Code generation; algorithmic problem solving.
MBPP	mbpp	Sanitized split of crowd-sourced Python programming tasks with test cases (Austin et al., 2021).	Code synthesis from natural language; test-driven correctness.
CRUXEval	cruxeval	800 Python functions with input prediction and output prediction tasks (Gu et al., 2024).	Code comprehension; execution tracing; I/O reasoning. **We use its input predition problem set which is the most difficult**
Character-Level & Spatial Reasoning			
CharBench	CharBench	A character-level benchmark with 175K instances across four tasks: character counting, frequency counting, and first/last occurrence finding (Uzan and Pinter, 2026).	Sub-token character perception; counting under tokenization; positional indexing.
StringBench	StringLLM	A string manipulation benchmark with 49 atomic tasks derived from Python and 1,462 composite tasks combining multiple operations (Wang et al., 2025).	String-level manipulation; character mapping; slicing and indexing; compositional string operations.
MazeBench	Maze-Bench	A spatial reasoning benchmark of 100 tokenized 5×5 mazes (Dao and Vu, 2025). Models must output a valid move list from origin to target without traversing walls.	Spatial reasoning; path planning; grid navigation; sequential action generation under constraints.

Table 7: Benchmarks used in our evaluation, grouped into four categories.

C.2 Training Datasets

We construct three categories of training data for both SFT and RL stages:

1. **`MIMIC`**: Our core synthetic dataset derived from competitive programming problems. Each sample consists of a story-wrapped reasoning problem paired with a step-by-step chain-of-thought (CoT) solution. The training data is curated through our multi-stage pipeline (Section A) and balanced across problem types, source difficulty levels, and problem identities. For SFT, this yields approximately 114K verified CoT samples; for RL, we select approximately 10K "useful" samples where the base model achieves partial correctness (1–7 out of 8 rollout samples correct) and balanced again by codeforces problem_id, capability requirement type and test case source, ensuring the problems are neither too easy nor too hard for effective policy gradient learning.

2. **Math**: For the SFT stage, we incorporate a widely-used open-source mathematical reasoning corpus, NuminaMath-CoT (LI et al., 2024), which provides high-quality chain-of-thought solutions for mathematical problems across diverse difficulty levels. For the RL stage, we use the official mathematical reasoning datasets provided by the Verl (Sheng et al., 2024) framework, which include curated prompts from GSM8K (Cobbe et al., 2021) and MATH (Hendrycks et al., 2021) training sets formatted for policy gradient training.

3. **Mixed**: A combination of `MIMIC` and Math data, constructed by randomly sampling equal portions from each source. This configuration is designed to study the complementary effects of code-based reasoning data and conventional mathematical reasoning data. As SFT suffers from significant catastrophic (Li et al., 2025; Chu et al., 2025) forgetting when using single-source data, we use the mixed dataset by default in SFT.

C.3 Supervised Fine-Tuning (SFT)

Framework. We use LLaMA-Factory (Zheng et al., 2024b) for all SFT experiments. Training is distributed across 8 GPUs using DeepSpeed ZeRO Stage-3 for memory-efficient full-parameter fine-tuning.

Hyperparameters. Table 8 summarizes the SFT training hyperparameters. All models share the same configuration except for model-specific chat templates (Qwen3 template for Qwen models, DeepSeek3 template for DeepSeek-R1-Distill-Llama-8B). For a fair comparison, we use checkpoints from approximately the same training step, around 1,500 steps, for evaluation.

Hyperparameter	Value
Fine-tuning type	Full parameter
Parallelism	DeepSpeed ZeRO-3
Number of GPUs	8
Per-device batch size	2
Gradient accumulation steps	8
Effective batch size	128
Learning rate	5×10^{-6}
LR scheduler	Cosine
Warmup ratio	0.1
Number of epochs	2
Max sequence length	8192
Precision	BF16
Gradient checkpointing	Enabled
Validation split	1%
Eval / save interval	200 steps

Table 8: SFT training hyperparameters.

Data Configurations. Recent studies have shown that SFT on single-domain data leads to catastrophic forgetting of general capabilities and overfitting to narrow output formats (Li et al., 2025; Chu et al., 2025). To mitigate this, we adopt a mixed training strategy rather than training exclusively on `MIMIC` data. Specifically, we randomly sample 114K samples from NuminaMath-CoT (LI et al., 2024) as mathematical reasoning data, matching the scale of our 114K `MIMIC` samples. Our primary training configuration (**Mixed**) combines equal portions from both sources, yielding approximately 114K training instances in total. This balanced mixture ensures that the model benefits from our code-mediated reasoning data while maintaining robust mathematical reasoning and general instruction-following capabilities. In ablation studies, we also train on Math-only configurations to verify that the mixed data outperforms single math domain source alone. And We also trained on original Codeforces problems (using LLM to replace the variables with real test cases, but without the story context) to illustrate the significance of our diverse contexts and synthesis.

C.4 Reinforcement Learning (GRPO)

Framework. We use Verl (Sheng et al., 2024) (Volcano Engine Reinforcement Learning for LLMs) for all RL experiments with Group Relative Policy with Optimization (GRPO) (Shao et al., 2024).

GRPO binary Reward Function. We use a rule-based reward function with flexible answer matching. For each prompt, the model generates $n = 8$ rollout responses. Each response is scored 1 (correct) or 0 (incorrect) by comparing the extracted answer against the ground-truth output produced by the AC code. The answer extraction pipeline applies three strategies in sequence: (1) exact string match, (2) substring containment check, and (3) numeric equivalence comparison (for mathematical answers).

Code-Instrumented Process Reward (CIR). In addition to the binary reward described above, we investigate a denser reward variant that leverages the multi-step structure of our code-synthesized problems. Since each training problem is derived from a competitive programming solution with well-defined intermediate execution checkpoints (e.g., input parsing, constraint validation, core algorithm steps, output formatting), we can assess *how far* a model's response progresses through the intended reasoning chain, even when the final answer is incorrect.

Concretely, we prompt a lightweight judge model (Qwen3-4B) with the model's response alongside the reference solution's intermediate outputs, and ask it to determine which step the response has reached. The reward is then computed as:

$$r = \frac{k}{K}, \tag{3}$$

where k is the number of correctly completed steps identified by the judge and K is the total number of steps in the reference solution. This provides a graded signal in $[0, 1]$ rather than a binary $\{0, 1\}$, enabling the policy to receive partial credit for partially correct reasoning trajectories.

To facilitate step-level evaluation across all reward strategies, we instruct the model to delimit its reasoning steps with explicit `[STEP]` markers during generation. This structured output format allows external PRMs to identify and score individual reasoning steps consistently.

Baseline PRM Configurations. For comparison, we evaluate two established process reward models under the same GRPO training setup:

- **PRM800K** (Zheng et al., 2024a; Zhang et al., 2025b): We use Qwen2.5-Math-7B fine-tuned on the PRM800K dataset, which contains approximately 800K step-level correctness annotations derived from GSM8K and MATH problems. At each `[STEP]` boundary, the model scores the reasoning step and we take the minimum step-level score as the trajectory reward, following standard PRM practice.
- **Math-Shepherd** (Wang et al., 2024a): We use the official `math-shepherd-mistral-7b-prm` checkpoint based on Mistral-7B. This model was trained on automatically generated step-level labels for mathematical reasoning. It scores each step by predicting the probability of a positive label (`+`) versus negative (`-`) at designated step boundaries (marked by a special Cyrillic delimiter token).

Both PRM models are hosted as inference services on a dedicated GPU and queried via HTTP during training. Importantly, both models were trained exclusively on mathematical reasoning data, which limits their ability to provide meaningful reward signals for the diverse, non-mathematical reasoning tasks in our `MIMIC` training set (see Table 5 for results).

Hyperparameters. Table 9 summarizes the GRPO training configuration. All experiments use on-policy rollouts generated by the vLLM engine (Kwon et al., 2023) with the same sampling parameters and we use the same step 190 for our evaluation for fair comparison.

Hyperparameter	Value
Algorithm	GRPO
Parallelism	FSDP
Number of GPUs	8
Rollout engine	vLLM
Rollout samples per prompt (n)	8
Train batch size	128
PPO mini-batch size	32
PPO micro-batch size per GPU	1
Actor learning rate	1×10^{-6}
KL loss coefficient	0.001
KL loss type	Low-variance KL
Max prompt length	4096
Max response length	4096
Precision	BF16
Gradient checkpointing	Enabled
Rollout GPU memory utilization	0.6

Table 9: GRPO training hyperparameters.

RL Dataset Configurations. We construct three RL dataset variants using the verl parquet format:

Data Selection for RL. We construct the `MIMIC` RL set from the synthesized data under a lexicographic objective. We first prioritize balance across `problem_id`, reasoning `type`, and test-case source, so that the selected set is as uniform as possible along these axes. Subject to this diversity constraint, we run 8 rollouts per instance and preferentially retain those whose empirical correctness is closest to 50%, since such cases maximize reward variance and yield the strongest policy gradient signal. We further encourage source diversity by preferring instances from different difficulty levels of the same problem.

D Distribution of Question Types in the Original Dataset

Table 10 summarizes the distribution of Codeforces problems across the five top-level categories in our first-stage taxonomy. The cleaned dataset contains 10,008 instances in total, and the distribution is markedly imbalanced. In particular, Type 5 (*Strict Rule & State Simulation*) and Type 3 (*Precise Mathematical Computation*) dominate the dataset, accounting for 46.02% and 40.64% of all instances, respectively; together, they cover 86.66% of the dataset. This indicates that Codeforces problems are primarily characterized by deterministic procedural reasoning and mathematically precise problem solving. By contrast, Type 2 (*Sub-Token/Character Manipulation*) represents a much smaller but still nontrivial portion (6.47%), while Type 4 (*Large-Scale Data Processing*, 4.80%) and Type 1 (*Text-Encoded Visual/Spatial Reasoning*, 2.07%) remain relatively underrepresented.

At the subtype level, several categories exhibit a clear head-heavy structure. Within Type 5, *Iterative State Simulation* is the largest subtype, whereas Type 3 is comparatively more balanced across arithmetic, algebraic, and set/statistical reasoning. Type 2 is strongly dominated by *Character Counting/Extraction/Replace*, and Type 4 is almost entirely concentrated in *Aggregation & Sorting*. Overall, these results suggest that Codeforces provides broad and dense coverage of symbolic, procedural, and mathematically grounded reasoning, but offers substantially less diversity in visual/spatial and layout-sensitive problem types.

This distribution is beneficial for both training and evaluation because it combines strong coverage of major reasoning types with explicit inclusion of smaller but capability-critical categories. In particular, although current models often underperform on string manipulation, precise computation, and text-encoded visual/spatial reasoning, our taxonomy ensures that each pre-defined fine-grained category is represented rather than omitted by the natural dominance of a few major types. As a result, the synthesized dataset provides broad capability coverage while still maintaining sufficient density in core categories such as mathematical computation and rule-based simulation, making it well suited for both scalable training and fine-grained diagnosis of model weaknesses.

To provide a more intuitive view of the scale and composition of our synthesized data, Figure 3 visualizes the hierarchical distribution of Codeforces problems in the Stage-I taxonomy. Although the distribution is naturally skewed toward several major reasoning types, the figure shows that all pre-defined categories are covered, including smaller but capability-critical ones. This broad coverage helps ensure that the synthesized training and test sets span diverse reasoning abilities rather than concentrating only on a few dominant domains. Notably, such coverage is achieved using only a single publicly available source, Codeforces, which suggests that our taxonomy and synthesis framework are practical under current web-scale data conditions. In other words, the fact that one easily accessible dataset already provides near-complete coverage highlights both the resource efficiency and the scalability of our approach, while relatively sparse categories can be further supplemented when actually needed.

E Preliminary Study of Problem's Difficulty and Code Rate

E.1 Difficulty-Dependent Effects Across Ratings

Rating	Original	Refined	Fused	Synthesized
< 1200	27.7	20.2	0.4	0.6
1200–1599	46.3	34.9	0.6	0.8
1600–1999	49.2	38.1	0.8	0.9
≥ 2000	44.9	35.5	0.8	0.7

Table 11: Code-writing rate in model reasoning across different Codeforces rating bands under different problem presentation formats. Lower is better.

Top-level Category	Subtype (abbrev.)	Count	Top-level Share (%)	Subtype Share (Parent, %)
Type 1: Text-Encoded Visual/Spatial	1.1 Grid/ASCII Parsing	135	2.07	65.2
	1.2 Geom./Rel. Grounding	69		33.3
	1.4 Layout-aware Tables/Diag.	2		1.0
	1.3 Glyph Decomp.	1		0.5
Type 2: Sub-Token/ Character	2.1 Char Count/Extract/Replace	527	6.47	81.3
	2.2 Char Mapping/Crypto.	61		9.4
	2.3 Formatting/Regex	60		9.3
Type 3: Precise Math Computation	3.3 Stats/Set Ops	1472	40.64	36.2
	3.1 High-Prec. Arithmetic	1430		35.2
	3.2 Algebra/Symbolic	1165		28.6
Type 4: Large-Scale Data Processing	4.2 Aggregation/Sorting	449	4.80	93.5
	4.1 Search/Filter	16		3.3
	4.3 Struct. Transform.	15		3.1
Type 5: Rule & State Simulation	5.3 Iterative Simulation	2206	46.02	47.9
	5.1 Pathfinding/Graphs	1468		31.9
	5.2 Constraints/Backtracking	932		20.2
Total	–	**10,008**	**100.00**	–

Table 10: Hierarchical distribution of the cleaned Stage-I dataset. "Top-level Share" is computed over all 10,008 instances, and "Subtype Share (Parent)" denotes the proportion within each top-level category. Abbreviated subtype names are used for space efficiency.

Rating	Original	Refined	Fused	Synthesized	Δ
<1200	46.6	55.1	48.6	72.3	-6.6
1200–1599	32.2	44.4	45.8	68.6	$+1.4$
1600–1999	19.3	30.6	41.4	65.1	$+10.8$
≥ 2000	13.0	24.4	40.6	56.9	$+16.2$

Table 12: Accuracy (%) across different Codeforces rating bands under different problem presentation formats. The last column Δ reports the accuracy gain of **Fused** over **Refined**.

We further analyze this effect across Codeforces difficulty ratings. Table 11 reports the proportion of cases in which the model attempted to write or reconstruct code during reasoning, and Table 12 reports the corresponding accuracy under different presentation formats. A clear trend emerges: for the **original** and **refined** formats, code-oriented behavior becomes much more frequent on harder problems, reaching nearly half of all cases in the 1200–1999 rating range. This suggests that as task difficulty increases, the model is increasingly likely to fall back on memorized competitive-programming patterns instead of directly reasoning about the answer. By contrast, both **fused** and **synthesized** formats keep the code rate consistently below 1% across all rating bands, showing that the reformulated presentation effectively suppresses this coding-oriented shortcut regardless of problem difficulty.

More importantly, the effect of reformatting is not uniform across difficulty levels. On lower-rated problems, the model's native reasoning ability is often already sufficient, so additional narrative packaging may introduce some noise and slightly reduce performance. However, on medium-to-hard problems, the benefit becomes increasingly significant: the performance gap between **fused** and **refined** grows steadily with rating, from negative on problems below 1200 to a substantial gain on problems above 2000. This pattern indicates that for difficult problems, the bottleneck is not simply insufficient reasoning capacity, but rather the model's tendency to choose the wrong strategy—namely, attempting to recall source problems or reconstruct code. Once this strategy is suppressed, the model performs markedly better by returning to direct reasoning. In this sense, our story-style reformulation acts as a form of *reasoning strategy correction*: instead of allowing the model to exploit coding priors, it forces the model onto a more faithful natural-language reasoning path.

These findings further clarify the role of our synthesis framework. The goal is not merely to paraphrase programming problems, but to redirect optimization away from competitive-programming recall and toward pure logical reasoning. This distinction is especially important for training: if original or lightly edited coding tasks are used directly, the model may improve mainly by strengthening coding-related retrieval and solution-template behavior. In contrast, our fused-and-structured



Figure 3: Hierarchical distribution of Codeforces problems in the Stage-I taxonomy. The inner ring denotes the five top-level categories and the outer ring denotes the corresponding fine-grained subtypes. Colors indicate top-level categories, while lighter shades represent their subtypes. Sector sizes are normalized within each level for visual clarity, and the actual numbers of instances are shown inside the segments.

synthesis pipeline encourages the model to solve problems through explicit reasoning over natural-language scenarios and carefully constructed test cases, which is better aligned with the capabilities we aim to evaluate and improve.

E.2 Performance Across Taxonomy Types

Type	Accuracy (%)	Total	Code Rate (%)
Visual/Spatial	37.2	2328	0.5
Char-Level	47.2	8045	1.2
Math	44.7	54121	0.9
Data Process	53.5	6011	0.5
Rule/State	37.2	57090	0.6

Table 13: Model performance across top-level taxonomy types under the fused setting.

We further analyze model performance across the major taxonomy types. As shown in Table 13, the model performs poorly across all categories, with especially low accuracy on **Visual/Spatial** and **Rule/State** reasoning, both at only 37.2%. These categories correspond to precisely the kinds of capability gaps that have repeatedly appeared in public discussions of strong language models: tasks such as determining left versus right in a mirror, comparing numbers like `9.9` and `9.11`, or counting the number of `r`'s in `strawberry`. What makes these problems difficult is not the need for broad world knowledge, but the need for *fine-grained symbolic fidelity* as careful structural interpretation, exact local manipulation, and faithful multi-step state tracking. Our results suggest that these weaknesses are not isolated anecdotes, but reflect broader and still unresolved deficiencies in current models.

To make these type-level difficulty patterns more concrete, we show three representative examples from the kinds of data synthesized by our pipeline (simplied for presentation in our paper). Although each example is simple for original program, all of them require the kind of fine-grained symbolic fidelity that remains fragile for current LLMs.

Example 5

Representative Examples of Fine-Grained Reasoning Tasks.

(1) Visual/Spatial Reasoning. A biologist tracks a lizard on a 2×2 grid whose cells are indexed by $(\text{row}, \text{column})$. The lizard starts at $(1, 2)$. The grid is then transformed in sequence: (i) rotate clockwise by 90° zero times, (ii) flip horizontally once, and (iii) rotate counterclockwise by 90° once. The question asks for the lizard's final position. This kind of task is closely related to popular failure cases such as mirror-based left/right reasoning: the challenge is not high-level knowledge, but exact spatial transformation under symbolic descriptions.

(2) Precise Numerical Comparison. A shop lists five prices:

$$17.02, \quad 7.18, \quad 7.8, \quad 10.1, \quad 10.05.$$

For each of the following budgets,

$$7.09, \quad 7.9, \quad 10.0, \quad 10.04,$$

the model must determine how many items have price less than or equal to the budget. This is a representative variant of the well-known `9.9` vs. `9.11` phenomenon: although the required reasoning is elementary, it depends on exact digit-level numerical comparison rather than approximate magnitude matching.

(3) Character-Level String Reasoning. A security system uses the trigger word

`silent` and scans the log entry `they are listening to the music`. The task is to determine whether the log contains a *contiguous substring* that is a permutation of `silent`. If such a substring exists, the correct output is `YES`; otherwise, `NO`. This type of example captures the same core weakness as counting letters in words like `strawberry`: the model must reason over exact character composition and local string structure, rather than relying on semantic similarity or approximate lexical cues.

This observation is exactly what motivates our data construction strategy. Rather than generating generic reasoning data, our method intentionally transforms competitive-programming problems into natural-language training instances that concentrate on those capability dimensions where programs are naturally reliable but language models remain fragile. In particular, visual/spatial tasks require models to reconstruct and manipulate text-encoded structures, while rule/state tasks require precise sequential simulation under strict constraints. Even categories such as **Math** and **Char-Level** remain far from saturated, showing that the resulting data targets fundamental reasoning weaknesses rather than narrow corner cases. Therefore, the type-level analysis supports the value of our approach as a *training data synthesis method*: it can systematically produce supervision signals for the very forms of precise reasoning that current LLMs still struggle to master.

E.3 Quality of Random Guided Synthetic Test Cases

Rating	Official	Level A	Level B	Level C
< 1200	52.4	51.8	48.3	42.8
1200–1599	44.8	51.9	48.0	38.4
1600–1999	36.1	50.7	45.1	33.1
2000–2399	31.5	52.0	46.0	32.2
≥ 2400	24.1	48.4	42.2	28.0

Table 14: Accuracy (%) across different test sources and Codeforces rating bands.

We also evaluate the quality of our automatically synthesized test cases produced by the code-to-text and key-point-guided test generation pipeline. Table 14 compares model accuracy on official tests and on our three synthesized levels. The results show that our generated test cases are consistently meaningful and non-trivial. In particular, **Level-C** often approaches the difficulty of the official tests and, for some rating ranges, remains broadly comparable to them. This is notable because our generated test cases are constructed under much more restricted settings as we only use relatively small constraints (e.g., $n \leq 50$ and values within 10^5), whereas official tests are not limited in this way and may include more adversarial large-scale cases. Despite this limitation, the generated tests still induce similar failure patterns, demonstrating that our method captures critical reasoning bottlenecks rather than relying solely on scale or brute-force stress. These findings support the practical value of our pipeline: even in the absence of official tests, code-guided synthesis can produce test cases that are sufficiently challenging for evaluation and training. This makes our synthesis pipeline more scalable.

F Token Distribution Shift

Following the methodology proposed by (Lin et al., 2024), we conducted a comprehensive analysis of token distribution shifts between the base and models after GRPO and SFT with our reasoning and math benchmarks. The analysis focuses on identifying and quantifying the changes in token prediction patterns that occur during the alignment process.

Our analysis procedure consists of the following steps:

1) For each position in the input text, we use the aligned model with greedy decoding to generate the output token o_t.

2) We then examine how this token is ranked in the base model's probability distribution P_{base}. This ranking, denoted as η, serves as our primary metric for categorizing token shifts.

3) Based on the base ranking η, we classify each token position into three categories:

- Unshifted positions ($\eta = 1$): The token is top-ranked in both base and trained models
- Marginal positions ($1 < \eta \leq 3$): The token has a relatively high probability in the base model
- Shifted positions ($\eta > 3$): The token is unlikely to be sampled by the base model

4) For shifted tokens, we calculate *Rank Improvement Ratio*: $\frac{\text{base_rank}}{\text{trained_rank}}$

Method	Unshifted	Marginal	Shifted
SFT vs Base	78.9%	15.6%	5.5%
GRPO vs Base	95.1%	4.8%	0.2%

Table 15: Shifted token distribution under different alignment methods, measured against the base model. We report the proportion of unshifted, marginal, and shifted positions.

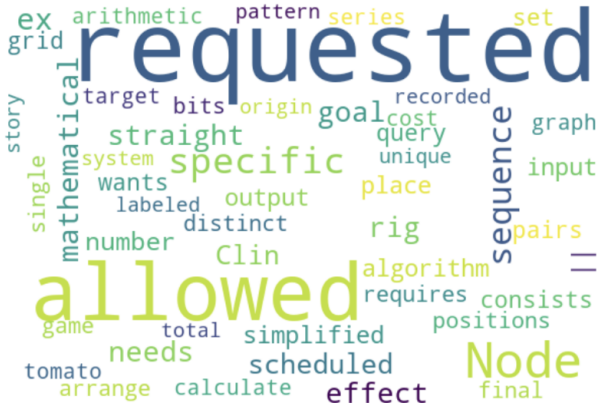


Figure 4: Word cloud of shifted tokens under SFT, measured relative to the base model. The shifted vocabulary is dominated by instruction- and description-level terms such as *requested*, *allowed*, *sequence*, *algorithm*, and *grid*, suggesting that SFT learns substantial surface-level regularities of the training data in addition to task behavior.

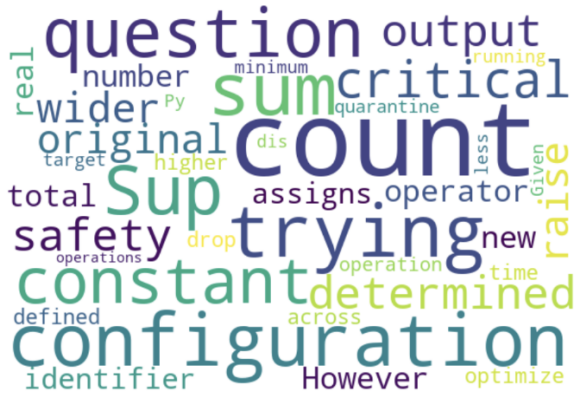


Figure 5: Word cloud of shifted tokens under GRPO, measured relative to the base model. Although the number of shifted positions is much smaller, the shifted vocabulary is more concentrated on reasoning-relevant terms such as *count*, *sum*, *configuration*, *operation*, and *determined*, indicating more targeted changes to computation and decision-making behavior.

Table 15 shows that both **SFT** and **GRPO** successfully modify the model's behavior on our synthesized training data, but they do so in very different ways. SFT causes a much broader redistribution of token preferences: only 78.9% of positions remain unshifted relative to the base model, while 5.5% become shifted. In contrast, GRPO is much more conservative, keeping 95.1% of positions unshifted and changing only 0.2% of positions in the shifted category. This indicates that both methods are effective, but GRPO achieves alignment with substantially smaller perturbations to the model's original token distribution.

The shifted vocabulary further reveals a qualitative difference between the two methods. As shown in Figure 4, 5, the shifted tokens under **SFT** are dominated by task-description and instruction-related words such as *requested* and *allowed*, as well as algorithmic or formatting-oriented terms such as *Node*, *sequence*, *algorithm*, *grid*, *bits*, *pairs*, *mathematical*, *simplified*, *output*, and *input*. This pattern suggests that SFT learns not only task-relevant behavior, but also the surface style of the training corpus—in particular, how to phrase solutions in benchmark-specific or instruction-like language. In other words, a substantial portion of SFT's token shift appears to reflect *what to say* and *how to present it*, rather than only *how to reason*.

By contrast, the very small set of shifted tokens under **GRPO** is much more tightly concentrated on computation and reasoning-critical vocabulary. Frequent examples include arithmetic and counting terms such as *count*, *sum*, *total*, and *number*; operational terms such as *configuration*, *operation*, *operator*, and *optimize*; and logical or mathematical descriptors such as *constant*, *determined*, *defined*, *critical*, *original*, and *real*. These are not merely stylistic markers: they are closely tied to the internal structure of calculation, constraint tracking, and decision-making. This suggests that GRPO is not primarily teaching the model *how to sound like* the training data, but rather making targeted adjustments to *how the model carries out reasoning and computation*.

This distinction helps explain the different downstream behaviors of the two alignment methods. Because SFT induces broader token-level shifts, including many shifts on formatting-sensitive and description-level vocabulary, it is more prone to over-adapting to the surface regularities of the training distribution. Such over-specialization can improve in-domain imitation while reducing flexibility on evaluation sets that differ in phrasing or problem presentation. GRPO, in contrast, achieves gains through minimal but strategically placed changes, preserving the base model's general lan-

guage competence while sharpening a small set of tokens that are disproportionately important for correct reasoning. In this sense, GRPO provides a more precise form of alignment: it does not broadly rewrite the model's language distribution, but selectively corrects the parts most relevant to mathematical and procedural reasoning.

G Training Dynamics

G.1 RL Training Dynamics Analysis

To understand how GRPO training with `MIMIC` data shapes model behavior, we analyze the training dynamics of our one of best-performing configuration (Qwen3-8B, $\mathcal{D}_{\text{mixed}}$) across 190 training steps. We examine four complementary perspectives: reward progression, policy divergence, response characteristics, and downstream benchmark evolution.

Reward progression. Figure 6a shows the mean reward steadily increasing from 0.61 to 0.81 over 190 steps. The learning curve exhibits two phases: a rapid improvement phase (steps 1–40) where the model quickly learns to solve easier problems, followed by a gradual refinement phase (steps 40–190) where gains become incremental as the model tackles harder problems. The absence of sudden jumps or collapses indicates stable optimization throughout training.

Policy stability. The KL divergence between the current policy and the reference model (Figure 6b) remains remarkably small throughout training, staying below 0.004 in smoothed values. A brief exploratory spike around steps 60–80 is followed by a natural decline, suggesting that the model temporarily explored alternative reasoning strategies before settling into a stable policy. This bounded KL divergence, combined with our low KL penalty coefficient ($\beta = 0.001$), confirms that the model achieves meaningful capability gains without deviating excessively from its pretrained distribution—a property further validated by our token-level shift analysis (Section F).

Response length evolution. The mean response length increases from approximately 850 to 1,330 tokens during training (Figure 6c). This growth is not indicative of *length hacking* which is a known failure mode where models generate verbose outputs to exploit reward signals, since our reward function evaluates only the final answer against a ground-truth output, rather than explicitly rewarding longer responses. Moreover, the increase in response length is gradual and stable over training, and is accompanied by a consistent rise in reward. Taken together, these trends suggest that the model progressively learns to produce more elaborate reasoning chains, allocating additional tokens to intermediate computation steps rather than superficial elaboration.

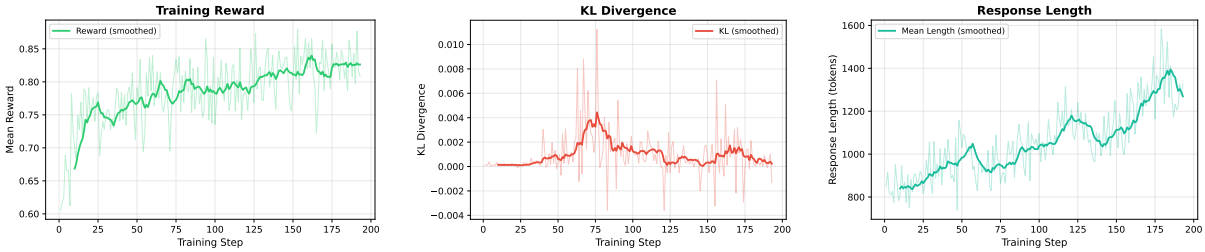
Downstream benchmark evolution. Figure 7 tracks four representative benchmarks across training steps using external evaluation. `MIMIC` demonstrates the most pronounced and consistent improvement (+9.8%), rising from 41.2% to 51.0% without signs of saturation. This steady climb, in contrast to the rapid convergence of mathematical benchmarks observed during validation, suggests that code-mediated reasoning abilities develop gradually through sustained policy refinement rather than early memorization.

Beyond the target task, three additional benchmarks exhibit clear positive transfer: BBH improves by +1.9%, reflecting enhanced complex reasoning; GSM-Plus gains +4.0%, indicating better mathematical generalization; and MBPP increases by +8.9%, demonstrating that strengthening reasoning capabilities can reciprocally benefit code generation. This cross-domain transfer supports our central thesis that code-mediated reasoning training produces broadly transferable cognitive improvements rather than narrow task-specific gains.

G.2 SFT Training Dynamics Analysis

To contextualize the advantages of reinforcement learning for code-mediated reasoning, we analyze the SFT training dynamics under the similar configuration and model configuration (Qwen3-8B, $\mathcal{D}_{\text{mixed}}$).

Rapid convergence and early saturation. Figure 8 reveals that SFT training loss drops sharply from 0.92 to approximately 0.45 within the first 400 steps, then gradually converges to 0.38 by the end of training (1,495 steps). The validation loss closely tracks the training loss, decreasing from 0.44 to 0.39, with minimal train-validation gap, indicating that the model is not heavily overfitting in the traditional sense. However, this rapid convergence masks a subtler issue: the model primarily learns to replicate the *surface format* of the training data rather than acquiring deeper reasoning capabilities, as evidenced by the benchmark dynamics.



(a) Training reward (b) KL divergence (c) Response length

Figure 6: **GRPO training dynamics** for Qwen3-8B on $\mathcal{D}_{\text{mixed}}$. (a) Mean reward increases steadily from 0.61 to 0.81, indicating consistent learning. (b) KL divergence remains bounded below 0.004 (smoothed), confirming stable policy updates without reward hacking. (c) Mean response length grows from ∼850 to ∼1,330 tokens, reflecting the emergence of more thorough reasoning chains.

Benchmark dynamics reveal shallow learning. Figure 9 tracks four representative benchmarks across SFT training steps. A consistent pattern emerges: rapid initial improvement within the first ∼800 steps, followed by plateaus or mild regression. ARC-Challenge gains +5.5% quickly as the model adapts to the answer format, while `MIMIC`—our primary target—improves by only +3.3% before saturating entirely.

This early saturation contrasts sharply with GRPO, where MIMIC accuracy rises continuously throughout training (+9.8%, Figure 7) without signs of plateauing. The gap is even more striking on code benchmarks: GRPO lifts MBPP by +8.9%, whereas SFT provides negligible code improvement. These results suggest that SFT primarily teaches the model to mimic the *surface format* of chain-of-thought demonstrations, while GRPO enables the model to discover and internalize genuine reasoning strategies that transfer across domains by optimizing for outcome correctness.

Why GRPO outperforms SFT for reasoning. The divergent training dynamics between SFT and GRPO illuminate a fundamental distinction in their learning mechanisms:

- **SFT saturates early**: all benchmarks converge within ∼800 steps, and the remaining training provides negligible improvement. The model learns the output distribution of the training data but does not develop novel reasoning strategies beyond what is explicitly demonstrated in the supervision signal.
- **GRPO improves continuously**: MIMIC accuracy rises monotonically throughout all GRPO steps without saturation (Figure 7). Because GRPO optimizes for *outcome correctness* rather than *token-level imitation*, the model is free to explore diverse reasoning paths, retaining only those that yield correct answers.
- **Transfer breadth**: SFT improvements are concentrated in format-sensitive benchmarks (ARC: +5.5%) while reasoning-heavy benchmarks improve modestly (MIMIC: +3.3%). GRPO shows balanced gains across both reasoning (MIMIC: +9.8%, BBH: +1.9%) and mathematical benchmarks (GSM+: +4.0%), alongside a surprising +8.9% improvement in code generation (MBPP), indicating deeper capability transfer.

These observations align with recent findings that SFT tends to memorize output formats while RL methods elicit genuine reasoning improvements (Chu et al., 2025). Our results further demonstrate that this distinction is particularly pronounced for code-mediated reasoning tasks, where the solution space is too diverse for format imitation to be sufficient.

H Task Type Ablation Analysis

To understand how each of the five task categories contributes to the final model performance, we conduct a systematic ablation study on Qwen3-8B using GRPO training. We design two complementary experiments:

- **Single-type training**: Train GRPO using only data from one task category (Type i only), producing five models.
- **Leave-one-out (LOO)**: Train GRPO using data from all categories *except* one (w/o Type i), producing five additional models.

All models are trained under identical hyperparameters and evaluated on the same benchmark suite. Full results are presented in Table 16.

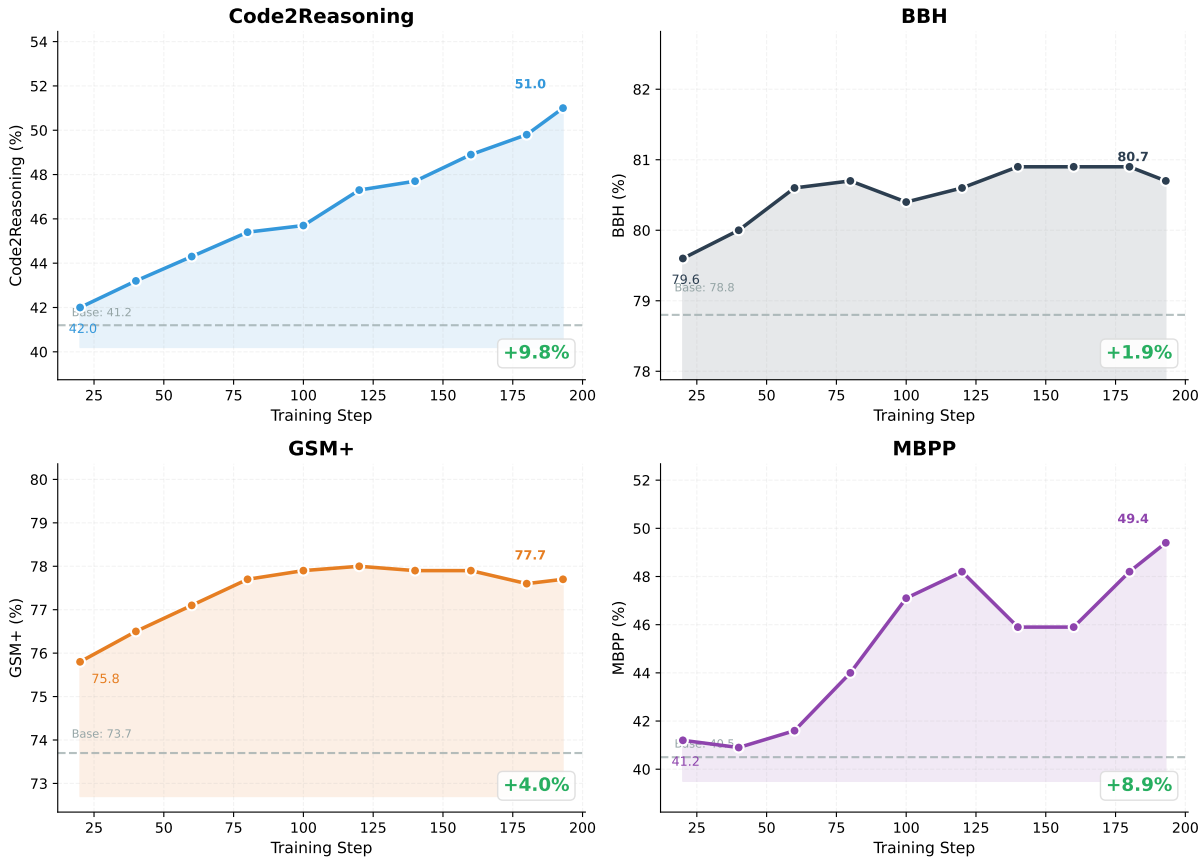


Figure 7: **Benchmark scaling during GRPO training** (Qwen3-8B, $\mathcal{D}_{\text{mixed}}$). All four benchmarks show consistent improvement over 190 training steps. Dashed lines indicate base model performance. MIMIC improves by +9.8%, demonstrating effective transfer from code-mediated reasoning to general problem-solving. Notably, MBPP (+8.9%) also improves substantially, suggesting that reasoning-focused training can reciprocally enhance code generation capabilities.

LOO Impact metric. To quantify the marginal contribution of each task category, we define the *LOO Impact* as the performance difference between the full model (trained on all five categories) and the leave-one-out model:

$$\Delta_{\text{LOO}}(\text{Type}_i, b) = \text{Score}_{\text{All}}(b) - \text{Score}_{\text{w/o Type}_i}(b), \quad (4)$$

where b is a benchmark or category average. A positive Δ_{LOO} indicates that removing Type i hurts performance—i.e., Type i is beneficial. A negative value indicates that removing it *improves* performance, suggesting potential interference. We visualize the LOO Impact across all benchmarks in Figure 11 and across category averages in Figure 10.

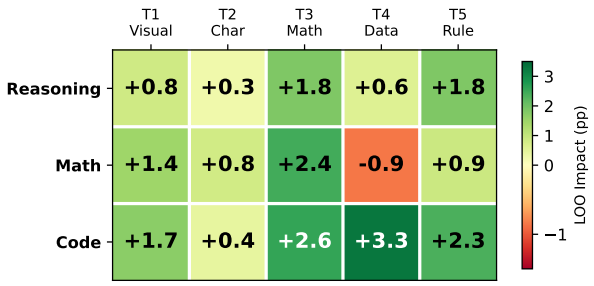


Figure 10: LOO Impact on category averages (Reasoning, Math, Code). Positive values (green) indicate beneficial contribution; negative values (red) indicate interference when included.

H.1 Per-Type Contribution Analysis

Type 3 (Precise Mathematical Computation) is the most broadly impactful category. Across all three evaluation dimensions, Type 3 shows the highest or near-highest LOO Impact: Reasoning +1.8, Math +2.4, Code +2.6 (Figure 10). The per-benchmark breakdown (Figure 11) reveals that this

Model	Setting	General Reasoning							Math					Code			
		ARC	BBH	MIMIC	GPQA	MMLU	DROP	Avg	GSM8K	GSM+	MATH	AIME	Avg	HEval	CruxE	MBPP	Avg
Qwen3-8B	Base	79.8	78.8	41.2	44.4	70.5	46.6	60.2	90.6	73.7	79.2	23.3	66.7	22.6	70.6	40.5	44.6
Qwen3-8B	Ours (All)	81.2	81.7	52.2	42.6	72.2	53.1	63.8↑3.6	91.7	78.1	81.0	26.7	69.4↑2.7	23.8	76.3	44.0	48.0↑3.4
(A) Single-Type Training: GRPO with only one task category																	
Qwen3-8B	Type 1 (Visual)	79.3	78.8	43.0	45.2	71.1	50.8	61.4↑1.2	90.3	74.6	79.9	26.7	67.9↑1.2	**22.9**	71.4	41.6	45.3↑0.7
	Type 2 (Char)	80.0	79.7	42.9	**46.1**	71.4	51.5	61.9↑1.7	92.0	75.4	81.1	**28.3**	69.2↑2.5	22.9	70.4	43.6	45.6↑1.0
	Type 3 (Math)	79.8	**81.8**	52.2	43.2	71.1	**52.9**	63.5↑3.3	90.8	78.6	81.1	23.3	68.5↑1.8	22.3	**75.0**	43.6	**47.0**↑2.4
	Type 4 (Data)	81.7	80.1	42.9	43.6	70.3	51.0	61.6↑1.4	90.5	75.3	80.8	16.7	65.8↓0.9	21.7	69.8	41.6	44.4↓0.2
	Type 5 (Rule)	**82.6**	81.6	**53.8**	39.4	**71.8**	52.4	**63.6**↑3.4	**92.3**	**78.8**	**81.2**	26.7	**69.7**↑3.0	22.3	72.0	**44.4**	46.2↑1.6
(B) Leave-One-Out: GRPO with all categories except one																	
Qwen3-8B	w/o Type 1	79.8	81.6	**52.6**	40.1	**72.1**	51.9	63.0↑2.8	**93.3**	**78.7**	79.6	20.0	67.9↑1.2	22.9	72.5	43.6	46.4↑1.8
	w/o Type 2	81.1	**82.2**	51.5	43.0	71.5	52.1	**63.6**↑3.4	91.7	78.1	81.0	23.3	68.6↑1.9	22.9	**75.9**	**44.0**	**47.6**↑3.0
	w/o Type 3	77.8	80.9	49.2	41.4	70.9	51.8	62.0↑1.8	91.7	77.4	**82.0**	16.7	67.0↑0.3	21.1	73.7	41.6	45.5↑0.9
	w/o Type 4	**81.6**	82.0	51.8	40.7	70.6	**52.7**	63.2↑3.0	91.7	78.5	80.7	**30.0**	**70.2**↑3.5	22.3	71.5	40.5	44.8↑0.2
	w/o Type 5	79.4	80.0	45.7	**43.4**	71.8	51.7	62.0↑1.8	90.9	76.1	80.3	26.7	68.5↑1.8	**23.6**	72.3	41.2	45.7↑1.1

Table 16: **Task Type Ablation (Qwen3-8B).** (A) *Single-type training*: GRPO trained with only one task category. (B) *Leave-one-out*: GRPO trained with all categories except one. Subscripts on **Avg** show absolute change vs. Base. **Bold** = best, underline = second best within each section.



Figure 8: **SFT training and validation loss** for Qwen3-8B on $\mathcal{D}_{\text{mixed}}$. The training loss drops sharply from 0.92 to ∼0.45 within the first 400 steps, then gradually converges to 0.39. The validation loss closely follows, plateauing at 0.39 with minimal train-validation gap.

contribution is driven by large gains on AIME (+10.0 pp), ARC (+3.4), MIMIC (+3.0), and CruxEval (+2.6). This aligns with our expectation: tasks involving high-precision arithmetic, algebraic manipulation, and numerical verification develop reasoning primitives (careful step tracking, exact computation) that transfer broadly across evaluation benchmarks. Notably, the +10.0 pp impact on AIME confirms that competition-level mathematics particularly benefits from training on precise computation tasks. In the single-type experiment, Type 3 alone achieves Reasoning Avg 63.5 and Math Avg 68.5, closely approaching the full model's 63.8 and 69.4, respectively.

Type 5 (Strict Rule & State Simulation) is the strongest single-type contributor to reasoning. When trained in isolation, Type 5 achieves the highest Reasoning Avg (63.6) and Math Avg (69.7) among all single-type models, and the highest single-type MIMIC score (53.8—surpassing even the full model's 52.2). The LOO analysis corroborates this: removing Type 5 causes the largest drop in MIMIC (-6.5 pp) and substantial drops in BBH (-1.7) and GSM+ (-2.0). Tasks in this category (graph traversal, constraint satisfaction, iterative state simulation) require the model to maintain and update complex state across multiple reasoning steps, a skill directly tested by multi-step reasoning benchmarks like BBH (tracking shuffled objects, logical deduction) and our MIMIC benchmark. The strong MIMIC drop is particularly meaningful since MIMIC specifically targets the abilities our data aims to develop.

Type 1 (Visual/Spatial Reasoning) has an outsized impact on AIME. While Type 1's average contributions are moderate (LOO: R+0.8, M+1.4, C+1.7), the per-benchmark view reveals a striking +6.7 pp impact on AIME—second only to Type 3. AIME competition problems frequently involve geometric reasoning, coordinate manipulation, and spatial visualization, which directly align with Type 1's training tasks (grid parsing, coordinate movement, geometric constraints). Type 1 also shows +3.8 pp on CruxEval, suggesting that spatial reasoning skills help with code execution tracing. The +2.5 pp impact on GPQA (graduate-level science questions) is also notable, likely reflecting the spatial reasoning demands of physics and chemistry problems.

Type 2 (Character-Level Operations) provides consistent but modest gains. Type 2 shows the smallest LOO Impact across all dimensions (R+0.3, M+0.8, C+0.4), suggesting that character-level ma-

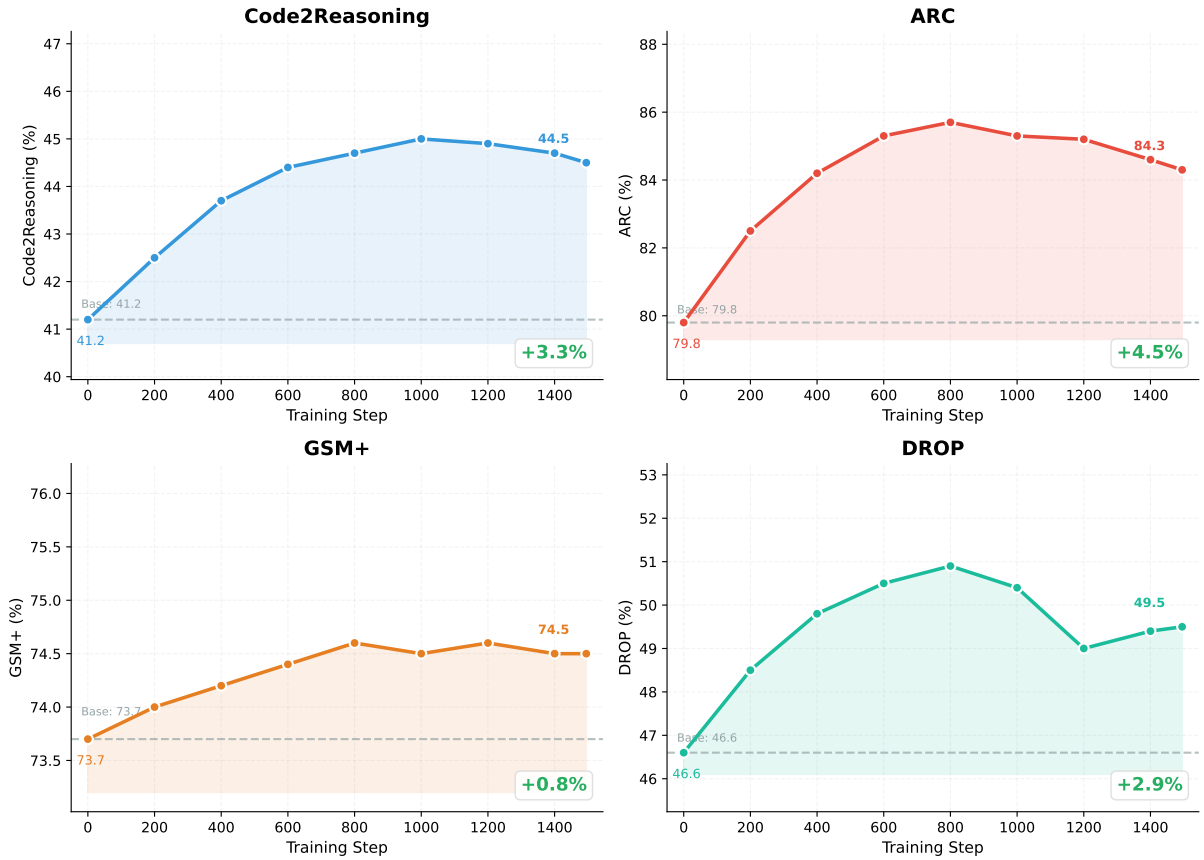


Figure 9: **Benchmark evolution during SFT** (Qwen3-8B, $\mathcal{D}_{\text{mixed}}$). Downstream benchmarks show quick initial gains that plateau or slightly decline after step 800, contrasting sharply with the continuous improvement observed under GRPO (Figure 7).

nipulation (counting, mapping, format validation) is the least critical category for standard benchmark performance. However, in the single-type experiment, Type 2 achieves the highest AIME score (28.3) among all single types, indicating that careful character-level attention may help with the precise symbolic manipulation required in competition mathematics. This category's contribution may also manifest more strongly on the specialized benchmarks (CharBench, StringBench) analyzed in Section 4.

Type 4 (Large-Scale Data Processing) shows a unique dual character. Type 4 exhibits the most interesting pattern: it has the *highest* LOO Impact on Code Avg (+3.3 pp) but a *negative* LOO Impact on Math Avg (-0.9 pp). The per-benchmark analysis reveals that this dual character stems from strong positive contributions to CruxEval (+4.8) and MBPP (+3.5), contrasted with a -3.3 pp impact on AIME. Tasks in this category involve search, filtering, aggregation, and structural transformation over large data, which are highly relevant to code comprehension and generation (CruxEval, MBPP) but may introduce noise for mathematical competition problems that require fundamentally different reasoning strategies. In the single-type experiment, Type 4 is the weakest single-type model (R: 61.6, M: 65.8, C: 44.4), performing below the base model on Math and Code averages, confirming that this category alone provides insufficient training signal. However, when mixed with other data, type 4 is crucial for coding ability.

H.2 Cross-Type Interactions and Complementarity

No single type matches the full mixture. While Types 3 and 5 individually approach the full model's Reasoning and Math averages, no single type matches the full model across *all three* dimensions simultaneously. The full model achieves (R: 63.8, M: 69.4, C: 48.0), whereas the best single-type results are (R: 63.6 by Type 5, M: 69.7 by Type 5, C: 47.0 by Type 3)—with no single type achieving more than two of these maxima. This demonstrates genuine complementarity: the full

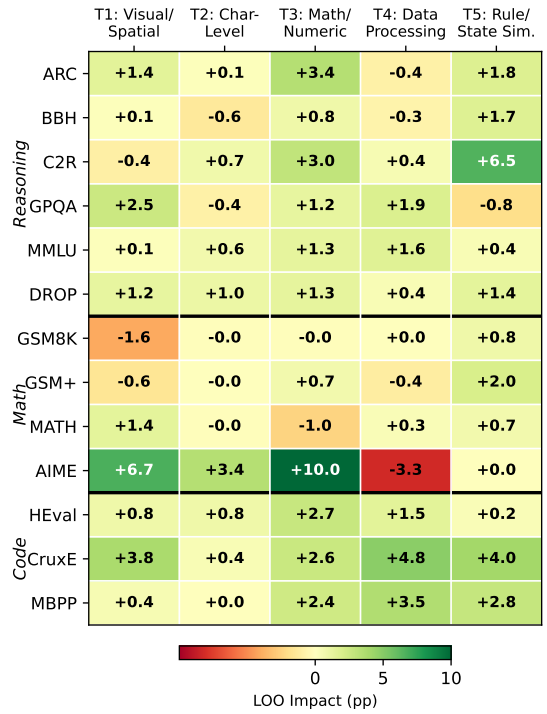


Figure 11: Per-benchmark LOO Impact. Each cell shows the accuracy drop (in percentage points) when the corresponding task type is removed from training. Horizontal lines separate the three evaluation categories: Reasoning, Math, and Code.

mixture benefits from diverse reasoning primitives that no single category can provide alone.

LOO Impacts are sub-additive. The sum of all five LOO Impacts exceeds the actual improvement of the full model over the base. For example, on Reasoning Avg, the sum of LOO impacts is $0.8 + 0.3 + 1.8 + 0.6 + 1.8 = 5.3$ pp, while the actual full model improvement is 3.6 pp. This sub-additivity indicates positive transfer and redundancy among types: each type partially compensates for others, and the marginal contribution of each type is smaller when others are already present. This is a healthy property—it means the training signal is robust to the removal of any single category.

Type 4's interference with Math is offset by Code gains. The negative LOO Impact of Type 4 on Math (-0.9 pp) might seem concerning, but removing Type 4 (w/o Type 4 model) actually achieves the highest Math Avg (70.2) among all configurations and even higher than the full model. This suggests that for purely mathematical performance, excluding large-scale data processing tasks may be optimal. However, including Type 4 provides the largest Code improvement (+3.3 pp), creating a natural trade-off. Our decision to include all five types in the full model prioritizes balanced performance across all evaluation dimensions over maximal performance on any single dimension.

H.3 Summary

The type ablation study yields three key insights: (1) All five task categories contribute positively to overall performance, with Types 3 (Precise Math) and 5 (Rule/State Simulation) being the most impactful; (2) Task diversity is essential. No single category can replicate the full mixture's balanced gains across reasoning, math, and code; (3) There exist meaningful cross-type interactions, such as Type 4's trade-off between code gains and math interference, that motivate our decision to include all categories in the final training mixture.

I Case Studies of Base Model and Trained Model

We present two representative examples comparing Qwen3-8B before and after GRPO training with our `MIMIC` data. These illustrate how training with code-synthesized data enables models to solve tasks that are trivial for programs but challenging for untrained LLMs.

Example 6

MazeBench — Spatial Navigation (Level: medium, 7-step solution)

Task: Navigate from S (2,1) to T (4,4) in a tokenized 5×5 maze:

```
+---+---+---+---+---+
|~~~~~~~~~~~|~~~~~~~|
+---+~~~+~~~+~~~+~~~+
|~~~~~~~|~~~~~~~|~~~|
+---+---+---+---+~~~+
|~~~~~S~~~~~~~~~|~~~|
+---+~~~+---+---+~~~+
|~~~~~~~|~~~~~~~~~~~|
+~~~+---+~~~+---+---+
|~~~~~~~~~~~~~~~~~T~|
+---+---+---+---+---+
```

Gold path:
(2,1)→(3,1)→(3,0)→(4,0)→
(4,1)→(4,2)→(4,3)→(4,4)

Qwen3-8B (Base): 13,831 characters of

reasoning. The model attempts multiple paths but repeatedly hits dead ends:
*"Move down to (3,1). Cell (3,1) has walls down_right. So moving down is **blocked**..."*
"From origin (2,1), move left to (2,0). Cell (2,0) has walls up_down_left. So moving left from (2,1) leads to..."
*"Wait, so from origin (2,1), moving down, left, or right all seem to lead to dead ends or loops. **Did I make a mistake here?**"*
→ **Exhausts token budget while cycling between failed paths. Only 2 move tokens extracted.**

Qwen3-8B + GRPO (Ours), 7,049 characters: The model follows a similar exploration process but proceeds more systematically. After parsing the maze, it methodically tests each direction:
"From (2,1), try right to (2,2). Walls: up_down. Try right to (2,3). Walls: up_down_right—blocked. Try left back...Not working."
"Let's try again from (2,1): move down to (3,1). Walls: down_right. Try left to (3,0)—allowed! From (3,0), move down to (4,0)—allowed!"
*"(4,0)→(4,1): right. (4,1)→(4,2): right. (4,2)→(4,3): right. (4,3)→(4,4): right. **Reached target!**"*
Correct!

Example 7

StringBench — Character-Level String Manipulation (Task: lstrip)

Task: Remove the leading characters in `"5f"` from `"5ff84790c47297a4dc5aee8a..."`.
Gold: `84790c47297a4dc5aee8a...` (3 characters removed)

Qwen3-8B (Base), 2,022 characters: The model misinterprets the semantics of the operation, treating "remove leading characters in `5f`" as "remove the two-character prefix `5f`":
"The first two characters are '5' and 'f'...removing the leading '5f' means removing the first two characters...the result is the rest of the string starting from index 2."
"So after removing first two characters, it becomes `f84790c47`.... I think that's the correct approach."
→ **Outputs `"f84790c47..."` — off-by-one error.** The correct behavior removes *all* leading characters belonging to the set {'5', 'f'}, stripping three characters (`5`, `f`, `f`) before the first non-matching character '8' while the model not systematically considering the next 'f' at all.

Qwen3-8B + GRPO (Ours), 258 characters: Correctly interprets the stripping semantics:
"To remove the leading characters '5f' from the string, we start from the third character (index 2) and take the rest of the string."
Output: `84790c47297a4dc5aee8a...`
→ **Correct!**

Discussion These examples reveal a consistent pattern: the untrained model attempts to solve these tasks through verbose reasoning (13K+ characters for a 7-step maze; 2K+ characters for a simple string operation), yet fails due to error accumulation, losing track of visited positions in the maze or miscounting character boundaries in the string.

These case studies reveal two systematic failure modes that our training addresses:

(1) Exploration inefficiency in structured search. On MazeBench, the base model correctly parses the maze structure but lacks a systematic traversal strategy. It explores paths in an ad-hoc manner, frequently revisiting dead ends and questioning its own reasoning ("*Did I make a mistake?*"). The GRPO-trained model exhibits similar exploration behavior but converges significantly faster as it abandons unproductive paths earlier and recognizes key transition points (e.g., "*try left to (3,0)—allowed!*") that unlock the solution. This mirrors the systematic backtracking characteristic of algorithms like DFS and BFS, precisely the type of reasoning patterns present in our training data.

(2) Failure to continue systematic exploration in precise string operations. On StringBench, the base model does not realize that solving the task requires examining the next character after

the stripped prefix. After removing the leading prompt `5f`, it stops the analysis instead of continuing to probe the following character `f`. By contrast, the trained model correctly understands the intended operation and proceeds with systematic exploration of the subsequent characters. This suggests that code-derived training examples improve the model's ability to follow the exact procedural semantics of string-manipulation tasks, rather than terminating early after a superficially plausible partial step.

Both failure modes share a common root: they require **precise, systematic computation** that programs handle trivially but that challenges the approximate, pattern-matching nature of language model inference. Our code-as-medium approach bridges this gap by training on data where such precision is guaranteed by construction.